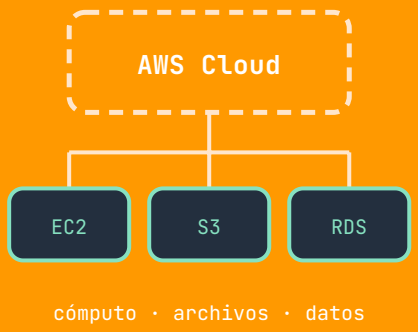
// capítulo de iniciación · computación en la nube · edición ampliada 2026

Introducción a las plataformas en la nube: AWS

Vas a entender **qué es la nube y por qué domina la industria**, vas a crear y proteger tu cuenta, y vas a construir con tus propias manos un servidor, un almacén de archivos, una base de datos y un backend en Node.js publicado en Internet. Todo explicado paso a paso, clic a clic y comando a comando.

Nivel: desde cero | 16 sesiones | 11 laboratorios guiados | 16 puntos de control | Proyecto final



cómputo · archivos · datos

Antes de empezar: cómo usar este capítulo

Este capítulo no se lee: **se hace**. Cada sesión combina tres capas. Primero la **intuición**, con una metáfora del mundo real. Luego el **concepto preciso**, con el vocabulario técnico que usan los ingenieros. Y por último la **práctica**, con pasos numerados que puedes seguir con la consola de AWS abierta en otra pestaña.

Avanza en orden. Las sesiones se apoyan unas en otras: el servidor que lanzas en la sesión 09 es el mismo donde instalas la web de la sesión 10, que se conecta a la base de datos de la sesión 12 y termina sirviendo el backend de la sesión 14. Si te saltas una, la siguiente no va a funcionar.

Al terminar vas a entender

Qué es la nube, los modelos IaaS/PaaS/SaaS, la responsabilidad compartida, regiones y zonas de disponibilidad, y cómo se cobra cada servicio.

Al terminar vas a saber hacer

Crear y blindar una cuenta, usar IAM y la CLI, lanzar EC2, publicar sitios en S3, crear bases en RDS y desplegar una API real.

Al terminar vas a poder decidir

Qué servicio usar para cada problema, qué región elegir, qué puertos abrir y cuáles jamás,

Al terminar vas a tener

Un proyecto funcional (API + base de datos + archivos) que puedes mostrar en tu portafolio

y cómo no pagar por lo que no usas.

y reconstruir cuando quieras.

Qué necesitas tener listo

- **Un computador** con Windows 10/11, macOS o Linux. Todos traen el comando `ssh` incluido, que vas a usar para entrar a tus servidores.
- **Un correo electrónico** que revises a diario. AWS envía ahí alertas de facturación y de seguridad; si no las lees, no sirven.
- **Un teléfono móvil** para verificar la cuenta por SMS y para instalar una app de autenticación (Google Authenticator, Microsoft Authenticator o Authy).
- **Una tarjeta de crédito o débito** habilitada para compras internacionales. AWS la usa para verificar tu identidad.
- **Un editor de código**, idealmente Visual Studio Code, y conocimientos básicos de terminal (moverte entre carpetas con `cd` y listar con `ls`).

> ❗ **Compromiso de seguridad y dinero**
>
> AWS es una plataforma profesional real: **si dejas recursos encendidos, te cobran**. Este capítulo te enseña a crear una alerta de gastos en la sesión 05 y a borrar todo en la sesión 16. Haz ambas cosas sin excepción. Un estudiante que entiende los costos vale más que uno que solo sabe hacer clic en "Launch".

Cómo leer los componentes del capítulo

> ℹ️ **Concepto**
>
> Una definición precisa que debes memorizar y saber explicar con tus palabras.

> ✅ **Buena práctica**
>
> Lo que hacen los equipos profesionales. Cópialo tal cual.

> ⛔ **Advertencia**
>
> Un error frecuente que cuesta dinero, seguridad o horas de depuración.

> 🧪 **Laboratorio**
>
> Práctica guiada con objetivo, tiempo estimado, pasos y resultado esperado.

Cuando veas algo así: `Consola › EC2 › Instances › Launch instances`, significa la ruta de clics dentro de la consola web. Los textos en **este estilo** son botones o campos exactos de la pantalla. La consola de AWS puede estar en español o en inglés; en este capítulo usamos los nombres en inglés porque son los que vas a encontrar en documentación, foros y trabajos.

SESIÓN 01 · CONCEPTO FUNDAMENTAL

¿Qué es exactamente la computación en la nube?

Alquilar capacidad de cómputo por Internet, en segundos y pagando por uso.

El problema que la nube resolvió

Imagina que en 2004 quieres lanzar una tienda en línea. Antes de escribir una línea de código tienes que **comprar servidores físicos**, esperar semanas a que lleguen, alquilar un espacio con aire acondicionado y electricidad de respaldo, conectar cables de red y contratar a alguien que los vigile de día y de noche. Si tu tienda fracasa, perdiste el dinero. Si tiene éxito y llegan diez veces más visitas de las previstas, los servidores colapsan y compras más... que llegan en otras tres semanas.

Ese modelo se llama **on-premises** ("en las instalaciones"): la infraestructura es tuya, está en tu edificio y todo el riesgo es tuyo. Tiene dos problemas graves. El primero es la **inversión inicial** (CAPEX, gasto de capital): pagas por adelantado, antes de saber si el proyecto funciona. El segundo es la **capacidad fija**: o compras de más y desperdicias dinero, o compras de menos y tu servicio se cae el día que más lo necesitas.

La computación en la nube invierte el modelo. Empresas como Amazon, Microsoft y Google construyen centros de datos gigantescos, y tú alquilas una fracción de esa capacidad **por Internet, en minutos y pagando solo por el tiempo que la usas**. El gasto pasa a ser operativo (OPEX): como la factura de la luz, pagas lo que consumes.

Definición formal

La **computación en la nube** es la entrega bajo demanda de recursos de TI (servidores, almacenamiento, bases de datos, redes, software) a través de Internet, con precios de pago por uso.

Tú pides recursos con un clic o una línea de código; el proveedor los aprovisiona automáticamente y los cobra por segundo, hora o gigabyte.

¿Sabías qué?

AWS nació dentro de Amazon. La empresa había construido una infraestructura enorme para su tienda y descubrió que podía venderla como servicio. En 2006 lanzó S3 (almacenamiento) y EC2 (servidores). Hoy Netflix, Airbnb y miles de bancos, universidades y startups funcionan sobre esa misma idea.

La metáfora: de la casa propia al hotel

Tener servidores propios es como **comprar una casa**: es tuya, pero pagas la hipoteca completa aunque viajes medio año, arreglas tú la tubería rota y, si llega toda la familia en diciembre, no puedes agregar diez habitaciones de un día para otro.

La nube es como **un hotel con capacidad casi infinita**: reservas una habitación esta noche, cien habitaciones para el congreso del viernes y ninguna el lunes. El hotel se encarga de la plomería, la electricidad y la seguridad del edificio. Tú solo pagas las noches que usaste. Y cuidado: el hotel no cierra con llave tu habitación por ti. Esa parte sigue siendo tu responsabilidad (volveremos a esto en unos minutos).

Las cinco características esenciales

El NIST (Instituto Nacional de Estándares y Tecnología de EE. UU.) define cinco características que todo servicio debe cumplir para llamarse "nube". Apréndelas: aparecen en exámenes y entrevistas.

Característica	Qué significa	Ejemplo en AWS
Autoservicio bajo demanda	Pides recursos tú mismo, sin llamar ni esperar a nadie.	Lanzas un servidor desde la consola en 60 segundos.
Amplio acceso por red	Accedes desde cualquier lugar con Internet y dispositivos estándar.	Administras tu cuenta desde el portátil o el celular.
Agrupación de recursos	El hardware se comparte entre muchos clientes de forma aislada y segura.	Tu servidor virtual vive en una máquina física que comparte con otros clientes, sin que se vean entre sí.
Elasticidad rápida	La capacidad crece y se reduce según la demanda, incluso de forma automática.	Auto Scaling pasa de 2 a 20 servidores en un pico de ventas y regresa a 2 después.
Servicio medido	Todo uso se mide y se cobra con precisión.	Pagas por segundo de servidor encendido y por gigabyte almacenado.

Los tres modelos de servicio: IaaS, PaaS y SaaS

No toda la nube es igual. La diferencia está en **cuántas capas administras tú y cuántas administra el proveedor**. Piensa en una pila de capas: red, servidores físicos, virtualización, sistema operativo, entorno de ejecución, aplicación y datos. Cuanto más arriba empieza el proveedor, menos trabajo tienes tú… y menos control.

Modelo	Qué alquilas	Tú administras	El proveedor administra	Ejemplos
IaaS Infraestructura como servicio	Máquinas virtuales, discos y redes	Sistema operativo, parches, runtime, aplicación, datos	Hardware, red física, virtualización, centro de datos	EC2, EBS, VPC
PaaS Plataforma como servicio	Un entorno listo para ejecutar tu código	Tu código y tus datos	Todo lo anterior + sistema operativo y runtime	Elastic Beanstalk, RDS, Lambda
SaaS Software como servicio	Una aplicación terminada	Tus datos y tu configuración de usuario	Absolutamente todo lo demás	Gmail, Office 365, Canva

🏗️ **IaaS = lote con casa en obra gris**

🛋️ **PaaS = apartamento amoblado**

Te entregan paredes y techo. Tú instalas pisos, cocina, muebles y cerraduras. Máximo control, máximo trabajo.

Llegas con tu maleta (tu código) y vives. No eliges el modelo de nevera, pero tampoco la reparas.

🏨 SaaS = habitación de hotel

Solo usas el servicio. No decides nada sobre el edificio; solo dónde dejas tus cosas.

⚡ Serverless = servicio a la habitación

Pides algo solo cuando lo necesitas y pagas por pedido. Si no pides nada, no pagas nada (Lambda).

Modelos de despliegue: pública, privada e híbrida

Además de "qué" alquilas, importa "dónde" vive la infraestructura:

- **Nube pública:** la infraestructura pertenece al proveedor y la comparten muchos clientes de forma aislada. AWS, Azure y Google Cloud son nubes públicas. Es el modelo de este capítulo.
- **Nube privada:** una infraestructura con características de nube pero dedicada a una sola organización, normalmente en su propio centro de datos. La usan algunos bancos y gobiernos con reglas estrictas.
- **Nube híbrida:** combina ambas. Por ejemplo, un hospital guarda historias clínicas en sus servidores y usa AWS para su portal web. Es muy común en empresas grandes que migran por etapas.

El modelo de responsabilidad compartida

Esta es la idea más importante de la sesión, y la que más estudiantes malinterpretan. En la nube **la seguridad se divide en dos**:

i AWS: seguridad DE la nube

Protege los edificios, los guardias, el hardware, la red física y el software que hace funcionar sus servicios. Nunca vas a entrar a un centro de datos de AWS, y no lo necesitas.

! Tú: seguridad EN la nube

Tus contraseñas, tus usuarios y permisos, los puertos que abres, los parches de tu sistema operativo, el cifrado de tus datos y quién puede leer tus archivos. **Si lo configuras mal, es tu culpa, no de AWS.**

La mayoría de filtraciones de datos en la nube no ocurren porque alguien "hackeó a AWS", sino porque un desarrollador dejó un almacén de archivos público, subió una contraseña a GitHub o abrió el puerto de la base de datos a todo Internet. Este capítulo te enseña a no cometer ninguno de esos errores.

! Antipatrón: ”está en la nube, entonces está seguro”

La nube no es segura por arte de magia: es **tan segura como tu configuración**. AWS te da las herramientas (cifrado, firewalls, permisos); usarlas bien es tu trabajo como ingeniero.

Ventajas y desventajas reales

Ventajas	Desventajas y riesgos
Sin inversión inicial: empiezas con centavos.	Costos inesperados si olvidas recursos encendidos.
Escalas en minutos, no en semanas.	Dependencia del proveedor (vendor lock-in) si usas servicios muy específicos.
Alcance global: servidores en decenas de países.	Requiere conexión a Internet estable.
El proveedor se encarga del hardware y del centro de datos.	Curva de aprendizaje: cientos de servicios y opciones.
Experimentas barato: si falla, apagas y listo.	Una mala configuración de seguridad expone datos a todo el mundo.

📌 Lo esencial de la sesión 01

- Nube = recursos de TI por Internet, bajo demanda y con pago por uso.
- IaaS te da máquinas; PaaS te da una plataforma; SaaS te da una aplicación terminada.
- AWS protege la infraestructura; tú proteges tu configuración, tus datos y tus accesos.

? Punto de control 01

a) Usas Gmail, despliegas tu código en Elastic Beanstalk y alquilas una máquina en EC2. Clasifica cada uno como IaaS, PaaS o SaaS.
b) Tu compañero deja una base de datos abierta a Internet y se filtran datos de clientes. ¿Es responsabilidad de AWS?

▼ Ver respuesta

a) Gmail = SaaS (solo usas la aplicación). Elastic Beanstalk = PaaS (subes código, AWS administra el entorno). EC2 = IaaS (administras el sistema operativo y todo lo de arriba).
b) No. Configurar el acceso a la red es seguridad *en* la nube, que corresponde al cliente según el modelo de responsabilidad compartida.

SESIÓN 02 · EL MERCADO

AWS, Microsoft Azure y Google Cloud: los tres gigantes

Quiénes son, en qué se diferencian y por qué empezamos con AWS.

Cuando alguien dice "vamos a la nube", la siguiente pregunta es **¿con cuál proveedor?** Hay decenas, pero tres concentran la mayoría del mercado mundial. Según los reportes trimestrales de la firma Synergy Research en 2025, AWS tenía cerca del 30 % del mercado de infraestructura en la nube, Microsoft Azure alrededor del 20 % y Google Cloud cerca del 13 %. Las cifras cambian cada trimestre, así que consulta el reporte más reciente si necesitas el dato exacto.

Criterio	AWS	Microsoft Azure	Google Cloud
Lanzamiento	2006 (S3 y EC2)	Anunciado en 2008, disponible en 2010	App Engine en 2008; plataforma completa después
Fortaleza principal	El catálogo más amplio (más de 200 servicios) y la mayor comunidad	Integración con Windows Server, Active Directory, Office 365 y .NET	Análisis de datos (BigQuery), Kubernetes e inteligencia artificial
Clientes típicos	Startups, empresas de todos los tamaños, gobiernos	Corporaciones que ya usan productos Microsoft	Empresas centradas en datos y ML
Servidor virtual	EC2	Virtual Machines	Compute Engine
Almacén de archivos	S3	Blob Storage	Cloud Storage
Base de datos gestionada	RDS	Azure SQL Database	Cloud SQL
Funciones serverless	Lambda	Azure Functions	Cloud Run functions

Lo que realmente importa

Fíjate en las últimas cuatro filas: **los tres proveedores ofrecen los mismos bloques con nombres distintos**. Si dominas los conceptos en AWS (servidor virtual, almacén de objetos, base gestionada, función serverless, red virtual), cambiar a Azure o Google es cuestión de aprender nombres nuevos, no de empezar desde cero.

¿Por qué empezamos con AWS?

1. **Es el más usado.** Encontrarás más ofertas de trabajo que piden AWS que cualquier otra nube.

2. **Tiene la comunidad más grande.** Casi cualquier error que te aparezca ya tiene respuesta en foros, blogs o en la documentación oficial.
3. **Sus certificaciones son reconocidas.** *AWS Certified Cloud Practitioner* es un primer paso realista para un estudiante (lo verás en la sesión 16).
4. **Ofrece un nivel gratuito** (Free Tier) pensado para aprender sin gastar, si lo usas con disciplina.

Otros proveedores que debes conocer

Existen proveedores más sencillos y económicos para proyectos pequeños, como DigitalOcean, Linode (Akamai) o Hetzner, y plataformas que esconden la infraestructura por completo, como Vercel, Netlify, Render o Railway. También hay nubes con fuerte presencia regional, como Oracle Cloud o Alibaba Cloud. No compiten en catálogo con los tres grandes, pero son excelentes para desplegar un proyecto personal rápido.

> ★ **¿Sabías qué?**
>
> Muchas empresas usan **multicloud**: por ejemplo, su aplicación principal en AWS y su análisis de datos en Google Cloud. Por eso los conceptos transferibles valen más que memorizar botones de una sola consola.

> ? **Punto de control 02**
>
> Una empresa colombiana usa Windows Server, Active Directory y Office 365 en todas sus oficinas. Otra es una startup de análisis de datos que entrena modelos de IA. ¿Qué proveedor tiene una ventaja natural para cada una? ¿Podría cualquiera de las dos usar AWS?
>
> ▼ **Ver respuesta**
>
> La primera tiene ventaja natural con **Azure** por la integración con productos Microsoft. La segunda podría preferir **Google Cloud** por BigQuery y sus herramientas de IA. Y sí: ambas podrían usar AWS sin problema, porque ofrece servicios equivalentes. La elección depende del contexto, el presupuesto y lo que ya sabe el equipo.

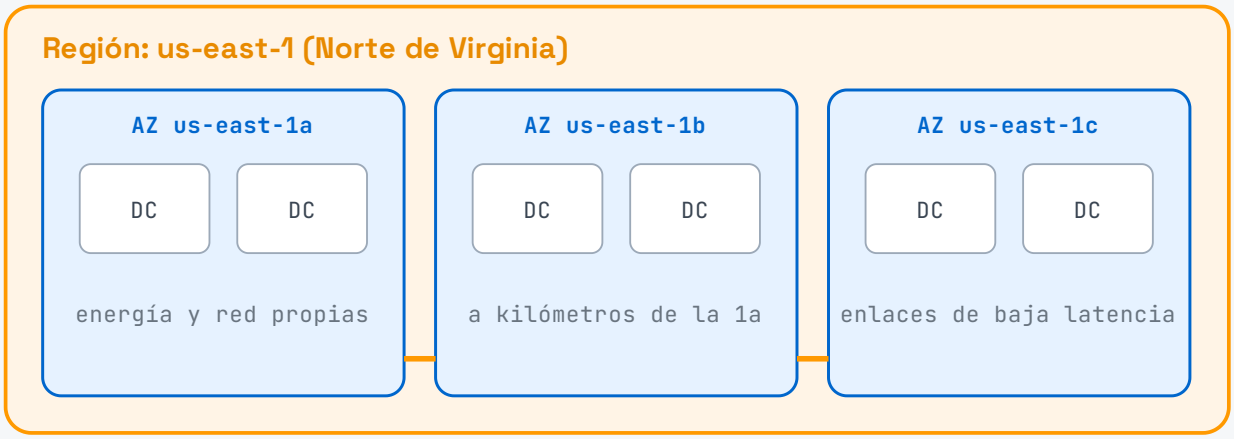


SESIÓN 03 · DÓNDE VIVE LA NUBE

Infraestructura global: regiones, zonas de disponibilidad y ubicaciones de borde

La nube no es etérea: son edificios reales en ciudades reales. Elegir bien dónde importa.

"La nube" suena abstracta, pero tus datos terminan en discos duros dentro de edificios concretos. AWS organiza esos edificios en una jerarquía de tres niveles. Entenderla te permite responder dos preguntas clave: **¿qué tan rápido responde mi aplicación a mis usuarios?** y **¿qué pasa si un centro de datos se incendia?**



Una región contiene varias zonas de disponibilidad (AZ); cada AZ tiene uno o más centros de datos (DC) con energía, refrigeración y red independientes.

Nivel	Qué es	Ejemplo	Para qué te sirve
Región	Una zona geográfica con varios centros de datos. Cada región es independiente de las demás.	`us-east-1` (Virginia), `sa-east-1` (São Paulo), `mx-central-1` (México)	Acercar tu app a tus usuarios y cumplir leyes sobre dónde se guardan los datos.
Zona de disponibilidad (AZ)	Uno o más centros de datos aislados dentro de una región, separados por kilómetros pero conectados con fibra de baja latencia.	`us-east-1a`, `us-east-1b`	Alta disponibilidad: si una AZ falla, tu app sigue viva en otra.
Ubicación de borde (Edge)	Puntos de presencia en cientos de ciudades que guardan copias en caché de tu contenido.	Usados por CloudFront (red de distribución de contenido)	Servir imágenes y videos más rápido a usuarios lejanos.

Cómo elegir una región: los cuatro criterios

1. **Latencia:** cuanto más cerca de tus usuarios, más rápida la respuesta. Para usuarios en Colombia, São Paulo, México o Virginia suelen ofrecer tiempos aceptables.
2. **Costo:** el mismo servicio cuesta distinto según la región. `us-east-1` suele estar entre las más económicas; `sa-east-1` suele ser más cara.
3. **Disponibilidad de servicios:** los servicios nuevos llegan primero a regiones grandes como `us-east-1`. No todas las regiones tienen todo.
4. **Normativa:** algunas leyes exigen que ciertos datos (salud, banca, gobierno) no salgan del país o del continente.

✓ Decisión para este capítulo

En todos los laboratorios usaremos `us-east-1` **(N. Virginia)**. Es de las más económicas, tiene todos los servicios y la mayoría de tutoriales la usan, así que tus pantallas se parecerán a las que encuentres en Internet. Para una aplicación real con usuarios latinoamericanos evaluarías `sa-east-1` o `mx-central-1` .

! El error que confunde a todos los principiantes

La consola muestra los recursos **de una sola región a la vez** (el selector está arriba a la derecha). Si creas un servidor en Virginia y luego miras la consola en São Paulo, **"desaparece"**. No se borró: estás mirando otra región. Antes de cualquier laboratorio, verifica que el selector diga `N. Virginia` .

★ ¿Sabías qué?

Algunos servicios son **globales**, no regionales: IAM (usuarios y permisos), Route 53 (DNS) y CloudFront. Por eso en la consola de IAM el selector de región dice "Global". Y los nombres de los buckets de S3 son únicos en todo el mundo, aunque el bucket viva en una región concreta.

? Punto de control 03

Despliegas tu aplicación en dos servidores, ambos en `us-east-1a` . Un corte eléctrico apaga esa zona. ¿Qué pasa con tu app? ¿Cómo lo habrías evitado?

▼ **Ver respuesta**

Tu app se cae por completo, porque ambos servidores dependían de la misma AZ. Debiste repartirlos en **AZ distintas** (por ejemplo, uno en `us-east-1a` y otro en `us-east-1b`) detrás de un balanceador de carga. Ese es el principio de **alta disponibilidad**.



SESIÓN 04 · PRIMEROS PASOS

Crear tu cuenta de AWS, paso a paso

Registro, verificación, elección de plan y primer ingreso a la consola.

Crear la cuenta toma entre 10 y 20 minutos. Hazlo con calma y con el teléfono a mano. Al terminar vas a tener un **usuario raíz (root user)**: la identidad dueña de la cuenta, con poder absoluto sobre todo. En la sesión 06 vas a aprender a guardarlo bajo llave y a trabajar con un usuario más seguro.

El nivel gratuito (Free Tier) y los planes de cuenta

Desde julio de 2025, al registrarte AWS te pide elegir entre dos planes. El **plan gratuito (Free plan)** incluye créditos promocionales (hasta 200 USD al momento de escribir este capítulo), dura un máximo de 6 meses o hasta que se agoten los créditos y **no te cobra en la tarjeta**; a cambio, algunos servicios costosos quedan bloqueados. El **plan pagado (Paid plan)** da acceso a todo y cobra lo que exceda los créditos. Además, muchos servicios tienen cuotas mensuales gratuitas permanentes (por ejemplo, un millón de invocaciones de Lambda al mes).

Las condiciones cambian con el tiempo: revisa siempre `aws.amazon.com/free` antes de registrarte. Si tu cuenta es anterior a julio de 2025, aplica el esquema clásico de 12 meses gratuitos con límites por servicio.

Recomendación para estudiantes

Elige el **plan gratuito**. Te protege de cobros accidentales mientras aprendes. Todo lo de este capítulo (EC2, S3, RDS, Lambda, IAM, VPC) funciona en ese plan. Cuando termine el período, AWS te pedirá pasar al plan pagado; si no lo haces, la cuenta se cierra y sus recursos se eliminan después de un tiempo de gracia.

El proceso completo de registro

1 Abre la página de registro

Entra a `https://aws.amazon.com` y haz clic en `Create an AWS Account` (o `Crear una cuenta` si la página está en español). Usa un navegador actualizado: Chrome, Edge, Firefox o Safari.

2 Correo del usuario raíz y nombre de la cuenta

Escribe tu correo en `Root user email address` y un nombre descriptivo en `AWS account name`, por ejemplo `estudiante-cloud-2026`. Haz clic en `Verify email address`.

> **¿Por qué importa?** Ese correo será el usuario raíz. Si pierdes acceso a él, recuperar la cuenta es muy difícil. Usa un correo personal que controles de por vida, no uno institucional que vence al graduarte.

3 Confirma el código de verificación

AWS envía un código de 6 dígitos a tu correo. Cópialo en el formulario y haz clic en `Verify`. Si no llega en 2 minutos, revisa la carpeta de spam.

4 Crea la contraseña del usuario raíz

Usa al menos 12 caracteres con mayúsculas, minúsculas, números y símbolos. Mejor aún: genera una con un gestor de contraseñas (Bitwarden, 1Password, el de tu navegador) y guárdala ahí.

> **¿Por qué importa?** Quien tenga esta contraseña puede lanzar cientos de servidores a tu nombre. Hay bots que buscan credenciales filtradas de AWS las 24 horas para minar criptomonedas.

5 Información de contacto

En `How do you plan to use AWS?` elige `Personal - for your own projects`. Completa nombre, teléfono con indicativo (+57 para Colombia), país, dirección, ciudad y código postal. Acepta el acuerdo de cliente y continúa.

6 Elige el plan de la cuenta

Selecciona `Free plan` si aparece la opción. Lee el resumen de lo que incluye y cuándo termina.

7 Información de pago

Ingresa los datos de tu tarjeta. AWS puede hacer una retención temporal de aproximadamente 1 USD para verificarla, que se reversa en unos días. Algunos bancos latinoamericanos bloquean este cargo internacional: si falla, llama a tu banco y pide habilitar compras internacionales en línea.

8 Verifica tu identidad por teléfono

Elige `Text message (SMS)`, confirma tu país y número, resuelve el captcha y escribe el código que recibes por mensaje.

9 Elige el plan de soporte

Selecciona `Basic support - Free`. Los planes de soporte pagados son para empresas; no los necesitas para aprender.

10 Ingresa por primera vez a la consola

Haz clic en `Go to the AWS Management Console`. En la pantalla de inicio de sesión elige `Root user`, escribe tu correo y contraseña. La activación completa puede tardar unos minutos (a veces hasta 24 horas); si un servicio dice que tu cuenta no está verificada, espera y reintenta.

> **Resultado esperado:** ves la página *Console Home* con un buscador en la parte superior, el selector de región a la derecha y tu nombre de cuenta en la esquina.

Recorrido por la consola

Antes de crear nada, ubica estas cinco zonas de la pantalla; las vas a usar en cada laboratorio:

Zona	Dónde está	Para qué sirve
Barra de búsqueda	Arriba al centro (atajo `Alt` + `S`)	Escribe "EC2", "S3" o "IAM" y entra directo al servicio. Es la forma más rápida de navegar.
Selector de región	Arriba a la derecha	Cambia la región que estás viendo. Déjalo en `N. Virginia (us-east-1)`.
Menú de la cuenta	Tu nombre, arriba a la derecha	Credenciales de seguridad, facturación, configuración de cuenta y cerrar sesión. Aquí ves el **ID de cuenta** de 12 dígitos.
CloudShell	Ícono de terminal `>_` en la barra superior	Una terminal Linux en el navegador con la CLI de AWS ya configurada (sesión 07).
Menú de servicios	Ícono de cuadrícula, arriba a la izquierda	Catálogo de todos los servicios agrupados por categoría.

🧪 Laboratorio 1 — Tu cuenta lista y reconocida

Objetivo: crear la cuenta, ingresar a la consola y dejarla configurada en la región de trabajo.

`⏱ 20 min` `$ 0 USD` `Dificultad: baja`

1 Registra tu cuenta

Sigue los 10 pasos anteriores. Anota el **ID de cuenta** (menú de tu nombre → número de 12 dígitos) en un lugar seguro.

2 Fija la región

En el selector de región elige `United States (N. Virginia) us-east-1`.

3 Explora con el buscador

Busca `EC2`, entra y regresa. Repite con `S3` e `IAM`. Fíjate que en IAM la región aparece como `Global`.

4 Marca tus favoritos

En los resultados del buscador haz clic en la estrella junto a EC2, S3, IAM, RDS, VPC y Billing and Cost Management. Aparecerán en la barra superior para acceso rápido.

5 Cambia el idioma si lo prefieres

Abajo a la izquierda (o en el ícono de engranaje) puedes cambiar el idioma de la consola. Te recomendamos dejarla en inglés para que coincida con este capítulo.

> **Resultado esperado:** cuenta activa, región us-east-1 seleccionada y seis servicios marcados como favoritos.

Reto extra: abre el menú de servicios y cuenta cuántas categorías hay. Elige una que no conozcas (por ejemplo, "Robotics" o "Quantum Technologies") y averigua para qué sirve uno de sus servicios.

? Punto de control 04

Terminaste el registro, entras a la consola, buscas EC2 y no ves la instancia que un compañero te dijo que creó "en tu cuenta" ayer. Menciona dos causas posibles.

▼ Ver respuesta

1) Estás mirando **otra región** distinta a la que usó tu compañero. 2) Tu compañero la creó en **su propia cuenta**, no en la tuya: cada cuenta de AWS está completamente aislada de las demás.



SESIÓN 05 · ANTES DE CREAR CUALQUIER COSA

Cómo cobra AWS y cómo protegerte con alertas de presupuesto

Configura tu red de seguridad financiera en 10 minutos.

La mayoría de sustos con AWS no vienen de hackers sino de **recursos olvidados**: un servidor que quedó encendido un mes, una base de datos que nadie borró. La regla profesional es simple: **primero configuras**

las alertas, después creas recursos.

Los tres ejes del cobro

Eje	Qué se mide	Ejemplos
Cómputo	Tiempo que un recurso está encendido (por segundo u hora)	Horas de una instancia EC2 o de una base RDS
Almacenamiento	Gigabytes guardados por mes	Archivos en S3, discos EBS, copias de seguridad
Transferencia de datos	Gigabytes que salen de AWS hacia Internet	Usuarios descargando imágenes o videos de tu app

Una particularidad que sorprende: **los datos que entran a AWS normalmente son gratis, los que salen se cobran**. También se cobran algunas cosas "invisibles": desde febrero de 2024 cada **dirección IPv4 pública** cuesta 0,005 USD por hora (unos 3,6 USD al mes), y un disco EBS sigue cobrando aunque su servidor esté apagado.

> ⏸️ **Apagar (Stop)**
> El servidor deja de cobrar horas de cómputo, pero su disco sigue guardado y cobrando almacenamiento.

> 🗑️ **Terminar (Terminate)**
> El servidor y (por defecto) su disco se eliminan. Deja de cobrar todo lo asociado.

Paso previo: permite que tu usuario administrador vea la facturación

Por defecto solo el usuario raíz ve la facturación, incluso si otro usuario tiene permisos de administrador. Como en la sesión 06 vas a dejar de usar el raíz, actívalo ahora:

1 **Abre la configuración de la cuenta**

Con el usuario raíz: menú de tu nombre → `Account`.

2 **Activa el acceso de IAM a la facturación**

Baja hasta `IAM user and role access to Billing information`, haz clic en `Edit`, marca `Activate IAM Access` y guarda con `Update`.

Crear un presupuesto con alertas (AWS Budgets)

```
Consola › Billing and Cost Management › Budgets › Create budget
```

1 Elige una plantilla

En **Budget setup** deja **Use a template (simplified)**. En plantillas elige **Zero spend budget**: te avisa apenas gastes el primer centavo por encima del nivel gratuito.

2 Nombra el presupuesto y pon tu correo

Nombre: `alerta-cero-gasto`. En **Email recipients** escribe el correo que revisas todos los días.

3 Crea el presupuesto

Haz clic en **Create budget**.

4 Crea un segundo presupuesto mensual

Repite el proceso con la plantilla **Monthly cost budget**, nombre `tope-mensual-5usd` y monto `5` USD. Esta plantilla te avisa cuando llegas al 85 % y al 100 % del monto, y cuando el pronóstico indica que lo vas a superar.

> **Resultado esperado:** dos presupuestos en la lista de Budgets con estado OK.

! Un presupuesto avisa, no apaga

AWS Budgets te **envía un correo**; no detiene tus recursos por sí solo. Si llega la alerta, entra de inmediato a la consola y busca qué está cobrando. Además, la facturación se actualiza con algunas horas de retraso, así que la alerta puede llegar después del gasto.

Herramientas para revisar tu gasto

- **Bills** (Facturas): detalle del mes actual por servicio y por región. Si ves una región que no usas, algo anda mal.
- **Free Tier**: en el panel de facturación muestra cuánto has consumido de las cuotas gratuitas y de tus créditos.
- **Cost Explorer**: gráficos del gasto por día y por servicio. Tarda hasta 24 horas en activarse la primera vez.
- **AWS Pricing Calculator** (`calculator.aws`): estima el costo mensual de una arquitectura *antes* de crearla.

🧪 **Laboratorio 2 — Tu red de seguridad financiera**

Objetivo: activar el acceso de IAM a la facturación, crear dos presupuestos y estimar un costo con la calculadora.

`⏱ 15 min` `$ 0 USD` `Dificultad: baja`

1 Activa el acceso de IAM a la facturación

Sigue los dos pasos de la sección anterior.

2 Crea el presupuesto de gasto cero

Plantilla **Zero spend budget** con tu correo.

3 Crea el presupuesto mensual de 5 USD

Plantilla **Monthly cost budget**.

4 Estima un servidor con la calculadora

Entra a `calculator.aws` → **Create estimate** → busca **Amazon EC2** → región us-east-1 → tipo `t3.micro` → uso 730 horas al mes (todo el mes encendido). Anota el costo mensual estimado.

5 Compara con uso parcial

Cambia el uso a 40 horas al mes (lo que un estudiante usa en clase) y compara. Así se ve en números el valor de apagar lo que no usas.

> **Resultado esperado:** dos presupuestos activos y una estimación que muestra la diferencia entre un servidor encendido todo el mes y uno encendido solo durante las prácticas.

Reto extra: en la calculadora agrega también un bucket S3 con 5 GB y una base RDS PostgreSQL `db.t4g.micro`. ¿Cuál de los tres servicios es el más costoso si todos quedan encendidos el mes completo?

? Punto de control 05

Apagaste (Stop) tu servidor hace 20 días, pero la factura sigue mostrando un pequeño cobro diario de EC2. ¿Qué lo explica?

▼ **Ver respuesta**

Un servidor detenido no cobra cómputo, pero su **disco EBS** sigue existiendo y se cobra por GB al mes. Si además le asignaste una **Elastic IP**, esa dirección pública también cobra por hora. Para dejar de pagar por completo, termina la instancia y libera la IP.

SESIÓN 06 · SEGURIDAD PRIMERO

IAM: identidades, permisos y cómo blindar tu cuenta

Guarda el usuario raíz bajo llave y trabaja siempre con el mínimo privilegio.

IAM (Identity and Access Management) es el servicio que responde a dos preguntas en cada acción que ocurre en tu cuenta: **¿quién eres?** (autenticación) y **¿tienes permiso para hacer esto?** (autorización). Cada clic en la consola y cada comando de la CLI pasa por IAM antes de ejecutarse.

La metáfora: el edificio de oficinas

Tu cuenta de AWS es un edificio. El **usuario raíz** es la llave maestra que abre todas las puertas, incluida la bóveda: se guarda en una caja fuerte y solo se usa en emergencias. Los **usuarios IAM** son las tarjetas de acceso de cada empleado. Los **grupos** son los departamentos (todo Contabilidad entra al piso 3). Las **políticas** son el reglamento que dice qué puertas abre cada tarjeta. Y los **roles** son credenciales de visitante: se prestan temporalmente a quien las necesite y se devuelven al salir.

Elemento	Qué es	Ejemplo
Usuario raíz	La identidad creada con el correo de registro. Tiene poder total y no se puede limitar.	tu.correo@gmail.com
Usuario IAM	Una identidad de larga duración para una persona o aplicación, con su contraseña o sus claves.	`estudiante-admin`
Grupo	Un conjunto de usuarios que comparten políticas. Se asignan permisos al grupo, no a cada persona.	`Administradores`, `Desarrolladores`
Política	Un documento JSON que permite o niega acciones sobre recursos.	`AdministratorAccess`, `AmazonS3ReadOnlyAccess`
Rol	Una identidad sin contraseña que se "asume" temporalmente. La usan servicios de AWS y aplicaciones.	Un servidor EC2 que lee archivos de S3 sin guardar claves
MFA	Autenticación de múltiples factores: además de la contraseña, un código de tu celular.	Google Authenticator, llave física, passkey

El principio de mínimo privilegio

Cada identidad debe tener **solo los permisos que necesita para su trabajo, y ninguno más**. Un script que sube respaldos a S3 no necesita permiso para borrar servidores. Si ese script se ve comprometido, el daño queda contenido a lo poco que podía hacer. Es la regla número uno de la seguridad en la nube.

Anatomía de una política IAM

Las políticas se escriben en JSON. Esta permite leer los archivos de un único bucket de S3 y nada más:

politica-lectura-fotos.json — JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "LeerFotosDeLaTienda",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::mi-tienda-fotos",
        "arn:aws:s3:::mi-tienda-fotos/*"
      ]
    }
  ]
}
```

1. **`Version`**: versión del lenguaje de políticas. Siempre `2012-10-17`; no es la fecha de hoy.
2. **`Sid`**: nombre opcional de la regla, para que otro humano entienda su propósito.
3. **`Effect`**: `Allow` (permitir) o `Deny` (negar).
4. **`Action`**: qué operaciones, con el formato `servicio:Operación`. `s3:*` significaría todas las de S3.
5. **`Resource`**: sobre qué recursos, identificados por su **ARN**. El primero es el bucket (para listarlo) y el segundo, `/*`, son todos sus objetos (para leerlos).

Cómo decide IAM: las tres reglas de evaluación

1. Todo está negado por defecto. Un usuario recién creado no puede hacer nada. **2. Un Allow explícito concede el permiso. 3. Un Deny explícito siempre gana**, aunque otra política diga Allow. Memoriza esto: resuelve el 90 % de los errores `AccessDenied`.

Un **ARN** (Amazon Resource Name) es el "número de cédula" de cada recurso: `arn:aws:servicio:región:id-de-cuenta:recurso`. En S3 la región y la cuenta van vacías porque los nombres de bucket son globales, por eso ves `arn:aws:s3:::nombre`.

Blindar el usuario raíz

`Consola (como raíz) › menú de tu nombre › Security credentials`

1 Instala una app de autenticación

En tu celular instala Google Authenticator, Microsoft Authenticator o Authy.

2 Asigna un dispositivo MFA

En **Multi-factor authentication (MFA)** haz clic en **Assign MFA device**. Nombre del dispositivo: `celular-root`. Tipo: **Authenticator app**. Clic en **Next**.

3 Escanea el código QR

Haz clic en **Show QR code** y escanéalo con la app. Aparece una entrada "Amazon Web Services" con un código de 6 dígitos que cambia cada 30 segundos.

4 Escribe dos códigos consecutivos

Escribe el código actual en **MFA code 1**, espera a que cambie y escribe el nuevo en **MFA code 2**. Clic en **Add MFA**.

> **¿Por qué dos códigos?** Así AWS confirma que el reloj de tu celular está sincronizado y que la app genera la secuencia correcta.

5 Verifica que no tienes claves de acceso del raíz

En la misma página, la sección **Access keys** debe estar vacía. El usuario raíz **nunca** debe tener claves de acceso.

> **Resultado esperado:** el dispositivo MFA aparece asignado. La próxima vez que ingreses como raíz, AWS te pedirá el código.

Crear tu usuario administrador de trabajo diario

Vas a crear un grupo con permisos de administrador y un usuario dentro de él. Desde ahora trabajarás con ese usuario y guardarás el raíz.

`Consola › IAM › User groups › Create group`

1 Crea el grupo Administradores

Nombre: `Administradores`. En **Attach permissions policies** busca y marca `AdministratorAccess`. Clic en **Create user group**.

2 Crea el usuario

Ve a **Users** → **Create user**. Nombre: `estudiante-admin`. Marca **Provide user access to the AWS Management Console**.

3 Elige el tipo de usuario

Si AWS pregunta si quieres usar IAM Identity Center, elige **I want to create an IAM user**. Identity Center es lo recomendado para organizaciones con muchas personas; para una cuenta personal de aprendizaje, un usuario IAM es suficiente.

4 Define la contraseña

Elige **Custom password** y crea una contraseña fuerte distinta a la del raíz. Puedes desmarcar **Users must create a new password at next sign-in**, porque el usuario eres tú mismo.

5 Agrega el usuario al grupo

En permisos elige **Add user to group** y marca `Administradores`. Clic en **Next** y luego en **Create user**.

6 Guarda la URL de inicio de sesión

La pantalla final muestra la **Console sign-in URL**, con la forma `https://123456789012.signin.aws.amazon.com/console`. Descarga el archivo `.csv` o copia la URL a tu gestor de contraseñas.

7 Crea un alias de cuenta (opcional)

En el panel de IAM, a la derecha, busca **Account Alias** → **Create**. Con el alias `mi-nube-2026` la URL queda `https://mi-nube-2026.signin.aws.amazon.com/console`, mucho más fácil de recordar que 12 dígitos.

✓ ¿Por qué AdministratorAccess y no algo menor?

En tu cuenta personal tú eres el administrador, y en la sesión 10 necesitarás crear roles de IAM (algo que la política `PowerUserAccess` no permite). En una empresa, en cambio, un desarrollador recibiría permisos mucho más acotados. La diferencia clave con el raíz: este usuario **se puede limitar, auditar, desactivar y borrar**; el raíz no.

! Antipatrones de identidad que debes evitar siempre

❌ Usar el usuario raíz para el trabajo diario. ❌ Compartir un mismo usuario entre varias personas. ❌ Crear claves de acceso para el raíz. ❌ Dar `AdministratorAccess` a un script "para que no falle". ❌ Pegar claves de acceso en el código, en un chat o en un repositorio de GitHub.

🧪 Laboratorio 3 — Cuenta blindada

Objetivo: proteger el raíz con MFA, crear tu usuario administrador con MFA y empezar a trabajar solo con él.

⏱ 25 min | $ 0 USD | Dificultad: media

1 MFA para el raíz

Sigue los 5 pasos de "Blindar el usuario raíz".

2 Grupo y usuario administrador

Sigue los 7 pasos de "Crear tu usuario administrador".

3 Cierra sesión como raíz

Menú de tu nombre → **Sign out**.

4 Ingresa como estudiante-admin

Abre tu URL de inicio de sesión (o elige **IAM user** en la pantalla de login y escribe tu ID de cuenta o alias). Usuario `estudiante-admin` y su contraseña.

5 Activa MFA también para este usuario

Menú de tu nombre → **Security credentials** → **Assign MFA device**. Repite el procedimiento con el nombre `celular-estudiante-admin`.

6 Comprueba tu identidad

El menú superior derecho debe mostrar `estudiante-admin @ 1234-5678-9012`. Entra a Billing: debes poder ver la facturación gracias a lo que activaste en la sesión 05.

> **Resultado esperado:** dos dispositivos MFA (raíz y administrador), y la consola abierta con el usuario IAM. A partir de aquí, todo el capítulo se hace con `estudiante-admin`.

Reto extra: crea un grupo `SoloLectura` con la política `ReadOnlyAccess` y un usuario `practicante` dentro de él. Ingresa con ese usuario en una ventana de incógnito e intenta crear un bucket de S3. Lee el mensaje de error: acabas de ver el mínimo privilegio en acción. Borra el usuario al terminar.

? Punto de control 06

Un usuario pertenece a un grupo con una política que permite `s3:*` sobre todos los buckets. Además tiene asignada directamente una política con `"Effect": "Deny"` para `s3:DeleteObject`. ¿Puede borrar archivos de S3? ¿Puede subirlos?

▼ Ver respuesta

No puede borrar: un Deny explícito siempre gana sobre cualquier Allow. **Sí puede subir** (`s3:PutObject`), porque está incluido en `s3:*` y ningún Deny lo bloquea.

SESIÓN 07 · MÁS ALLÁ DE LOS CLICS

La línea de comandos de AWS: CloudShell y AWS CLI

Lo que haces con diez clics, lo haces con un comando que puedes repetir y automatizar.

La consola web es ideal para aprender y explorar, pero los profesionales automatizan. La **AWS CLI** (Command Line Interface) te permite controlar casi todos los servicios con comandos de texto. Un comando se puede copiar, documentar, meter en un script y ejecutar mil veces sin equivocarse; una secuencia de clics, no.

Todos los comandos siguen la misma estructura:

estructura CLI

```
aws <servicio> <operación> --opción valor
aws s3 ls                                  # servicio s3, operación ls
aws ec2 describe-instances --region us-east-1
```

Opción A (recomendada para empezar): AWS CloudShell

CloudShell es una terminal Linux que se abre dentro del navegador, con la CLI ya instalada y autenticada con el usuario con el que entraste a la consola. No instalas nada ni creas claves. Incluye 1 GB de almacenamiento persistente por región para tus archivos.

1 Abre CloudShell

Haz clic en el ícono `>_` de la barra superior de la consola (o busca "CloudShell"). La primera vez tarda uno o dos minutos en preparar el entorno.

2 Pregúntale a AWS quién eres

Escribe el siguiente comando y presiona `Enter`:

CloudShell BASH

```
aws sts get-caller-identity
```

```
salida →
{
    "UserId": "AIDA4EXAMPLEEXAMPLE7",
    "Account": "123456789012",
    "Arn": "arn:aws:iam::123456789012:user/estudiante-admin"
}
```

> **Cómo leer la salida:** `Account` es tu ID de cuenta y `Arn` confirma que estás actuando como `estudiante-admin`. Si ahí dice `root`, cerraste mal la sesión del raíz.

Opción B: instalar la AWS CLI en tu computador

Instalarla localmente te permite usarla desde tu editor y tus propios scripts. Descarga siempre la **versión 2**:

Sistema	Cómo instalar
Windows	Descarga y ejecuta el instalador `https://awscli.amazonaws.com/AWSCLIV2.msi`. Luego abre una nueva ventana de PowerShell.
macOS	Descarga y ejecuta `https://awscli.amazonaws.com/AWSCLIV2.pkg`, o con Homebrew: `brew install awscli`.
Linux	Ejecuta los comandos del bloque siguiente.

terminal (Linux x86_64) BASH

```bash
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws --version        # comprueba la instalación
```

salida →

```
aws-cli/2.x.x Python/3.x Linux/6.x exe/x86_64
```

Crear claves de acceso para la CLI local

Para que la CLI de tu computador actúe en tu nombre necesita un par de **claves de acceso**: un *Access Key ID* (como un nombre de usuario, empieza por `AKIA`) y un *Secret Access Key* (como una contraseña). Son muy poderosas: quien las tenga puede hacer todo lo que tu usuario puede hacer.

`Consola › IAM › Users › estudiante-admin › Security credentials › Create access key`

1 Elige el caso de uso

Selecciona **Command Line Interface (CLI)**. AWS mostrará una recomendación de usar alternativas (como CloudShell o IAM Identity Center); marca la casilla de confirmación y continúa.

2 Agrega una etiqueta descriptiva

Por ejemplo `portatil-personal-2026`, para saber luego dónde está guardada cada clave.

3 Copia las claves una sola vez

El *Secret Access Key* solo se muestra en esta pantalla. Cópialo directamente al siguiente paso; no lo pegues en notas, correos ni chats.

4 Configura la CLI

En tu terminal ejecuta `aws configure` y responde las cuatro preguntas:

terminal BASH

```
aws configure
AWS Access Key ID [None]: AKIA..............
AWS Secret Access Key [None]: ********************************
*****
Default region name [None]: us-east-1
Default output format [None]: json
```

> **¿Dónde queda guardado?** En dos archivos de tu carpeta de usuario: `~/.aws/credentials` (las claves) y `~/.aws/config` (región y formato). Esos archivos jamás se suben a un repositorio.

5 Verifica

Ejecuta `aws sts get-caller-identity`. Debe mostrar el ARN de `estudiante-admin`.

! Si alguna vez filtras una clave

Si una clave llega a GitHub, a una captura de pantalla o a un chat, asume que ya la tienen otras personas: hay bots que rastrean repositorios públicos en busca de claves que empiezan por `AKIA`. Entra de inmediato a IAM → tu usuario → `Security credentials`, **desactiva** la clave, **elimínala**, crea una nueva y revisa la facturación y las instancias de *todas* las regiones.

Tus primeros comandos útiles

terminal o CloudShell — BASH

```bash
# ¿Quién soy y en qué cuenta estoy?
aws sts get-caller-identity

# Listar todas las regiones disponibles en formato tabla
aws ec2 describe-regions --output table

# Listar tus buckets de S3 (por ahora, ninguno)
aws s3 ls

# Listar tus instancias EC2 mostrando solo ID y estado
aws ec2 describe-instances --query "Reservations[].Instances[].[InstanceId,State.Name]" --output table

# Ver la ayuda de cualquier comando
aws s3 help
```

✓ Buenas prácticas con la CLI

Usa `--output table` para leer y `--output json` para procesar con scripts. Usa `--query` para filtrar solo los campos que te interesan. Si manejas varias cuentas, crea perfiles con `aws configure --profile nombre` y úsalos con `--profile nombre`. Y antes de un comando destructivo, ejecuta `aws sts get-caller-identity` para confirmar en qué cuenta estás.

? Punto de control 07

Ejecutas `aws s3 ls` en tu portátil y obtienes `Unable to locate credentials`. En CloudShell el mismo comando funciona. ¿Por qué?

▼ Ver respuesta

En tu portátil la CLI todavía no tiene claves configuradas: te falta ejecutar `aws configure` (o el archivo `~/.aws/credentials` no existe). CloudShell, en cambio, hereda automáticamente las credenciales de la sesión de consola con la que lo abriste.

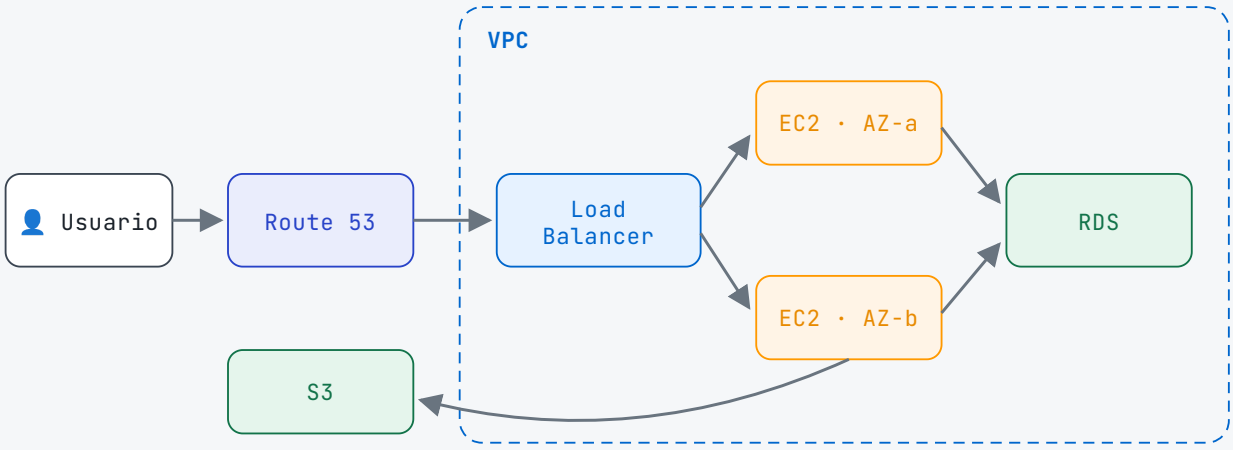
SESIÓN 08 · EL MAPA DEL TERRITORIO

Los servicios esenciales de AWS y cómo encajan

Más de 200 servicios, pero un puñado resuelve la mayoría de aplicaciones.

El catálogo de AWS intimida. La buena noticia: casi toda aplicación web se construye con las mismas piezas. Si entiendes **qué problema resuelve cada una**, puedes diseñar la arquitectura de un proyecto real. Piensa en una tienda en línea y mira qué servicio cubre cada necesidad:

Necesidad de la tienda	Servicio	Categoría	Analogía
Ejecutar el código del backend	**EC2**	Cómputo	Un computador alquilado
Ejecutar tareas cortas cuando pasa algo	**Lambda**	Cómputo serverless	Un mensajero que solo cobra por encargo
Guardar fotos de productos y facturas en PDF	**S3**	Almacenamiento de objetos	Una bodega infinita de cajas etiquetadas
El disco duro del servidor	**EBS**	Almacenamiento en bloque	El disco duro interno de un PC
Guardar clientes, pedidos y productos	**RDS**	Base de datos relacional	Un archivador con fichas ordenadas y un bibliotecario incluido
Guardar carritos y sesiones a gran escala	**DynamoDB**	Base de datos NoSQL	Un casillero gigante con acceso instantáneo por número
Aislar todo en una red privada	**VPC**	Redes	El conjunto cerrado con portería
Controlar quién puede hacer qué	**IAM**	Seguridad	Las tarjetas de acceso del edificio
Ver métricas, registros y alarmas	**CloudWatch**	Monitoreo	Las cámaras y sensores del edificio
Traducir `mitienda.com` a una IP	**Route 53**	DNS	La guía telefónica de Internet
Servir imágenes rápido en todo el mundo	**CloudFront**	Red de entrega de contenido	Sucursales con copias del catálogo en cada ciudad
Repartir tráfico entre varios servidores	**Elastic Load Balancing**	Redes	El anfitrión que asigna mesas en un restaurante



Arquitectura clásica de una aplicación web en AWS. En las próximas sesiones vas a construir una versión simplificada: una sola instancia EC2, RDS y S3.

Tipos de almacenamiento: no los confundas

Bloque (EBS)
Un disco duro conectado a **un** servidor. Rápido, para el sistema operativo y bases de datos instaladas a mano.

Objetos (S3)
Archivos completos con una URL, accesibles desde cualquier lugar por HTTP. Ilimitado y muy durable.

Archivos (EFS)
Una carpeta de red compartida que **varios** servidores montan a la vez.

Base de datos (RDS / DynamoDB)
Datos estructurados que consultas y filtras, no archivos sueltos.

? Punto de control 08

Para cada caso, elige el servicio: **a)** los videos que suben los usuarios de tu app; **b)** los datos de usuarios con sus pedidos y pagos; **c)** redimensionar cada imagen justo después de que se sube; **d)** el backend Express que corre las 24 horas.

▼ Ver respuesta

a) S3. **b)** RDS (datos relacionales con integridad entre tablas). **c)** Lambda, activada por el evento de subida a S3. **d)** EC2 (o un servicio de contenedores más adelante).

SESIÓN 09 · TU PRIMER SERVIDOR

EC2 a fondo: lanzar, conectar y administrar una máquina virtual

Elastic Compute Cloud: un servidor Linux en Internet, listo en un minuto.

EC2 te alquila máquinas virtuales llamadas **instancias**. Una instancia es un computador completo: tiene procesador (vCPU), memoria RAM, disco, tarjeta de red y sistema operativo. La diferencia con tu portátil es que no la ves ni la tocas; la administras a distancia por **SSH**, un protocolo que abre una terminal segura sobre Internet.

Vocabulario obligatorio

Término	Qué es	Analogía
Instancia	Una máquina virtual en ejecución.	El computador alquilado.
AMI (Amazon Machine Image)	La plantilla desde la que nace la instancia: sistema operativo y software preinstalado.	La imagen de instalación de Windows o Ubuntu.
Tipo de instancia	La combinación de vCPU, RAM y red.	El modelo del computador.
Par de claves (Key pair)	Una clave pública (queda en el servidor) y una privada (el archivo `.pem` que descargas).	La cerradura y su única llave.
Grupo de seguridad	Firewall virtual que decide qué tráfico entra y sale.	El portero con lista de invitados.
Volumen EBS	El disco de la instancia.	El disco duro.
User data	Un script que se ejecuta automáticamente en el primer arranque.	La lista de tareas del primer día.

Cómo leer el nombre de un tipo de instancia

t3.micro — TIPO

```
1 t       → familia
2 3       → generación
3 micro   → tamaño
4
5   t4g.micro   → "g" = procesador Graviton (ARM)
6   m7i.large   → "i" = procesador Intel
7   c7g.xlarge  → familia c: mucho cómputo
```

1. **Familia**: `t` = uso general con ráfagas (ideal para aprender y apps pequeñas), `m` = uso general balanceado, `c` = mucho cómputo, `r` = mucha memoria, `g` / `p` = GPU.
2. **Generación**: número más alto = hardware más nuevo y normalmente mejor precio por rendimiento.
3. **Tamaño**: `nano` < `micro` < `small` < `medium` < `large` < `xlarge`. Un `t3.micro` tiene 2 vCPU y 1 GiB de RAM.

> **Instancias con ráfagas (familia t)**
>
> Las instancias `t` acumulan **créditos de CPU** mientras están poco usadas y los gastan cuando necesitan potencia. Son perfectas para una web con tráfico bajo o intermitente. Si tu aplicación usa el 100 % de CPU todo el tiempo, una familia `m` o `c` te sirve mejor.

Paso a paso: lanzar tu primera instancia

`Consola › EC2 › Instances › Launch instances`

1 Nombre y etiquetas

En `Name` escribe `servidor-lab`. Esto crea la etiqueta `Name`, que te ayuda a reconocer la instancia en la lista.

2 Imagen del sistema operativo (AMI)

En `Application and OS Images` elige `Ubuntu` y luego `Ubuntu Server 24.04 LTS`, arquitectura `64-bit (x86)`. Verifica la etiqueta `Free tier eligible`.

> **¿Por qué Ubuntu?** Es la distribución Linux con más tutoriales y la que probablemente ya viste en clase. Su usuario por defecto es `ubuntu`. Si eliges Amazon Linux 2023, el usuario es `ec2-user` y el gestor de paquetes es `dnf` en lugar de `apt`.

3 Tipo de instancia

Elige el tipo marcado como `Free tier eligible`, normalmente `t3.micro` (o `t2.micro` en algunas regiones y cuentas antiguas).

4 Par de claves

En **Key pair (login)** haz clic en **Create new key pair**. Nombre: `clave-lab`. Tipo: **ED25519** (o RSA si usas software antiguo). Formato: **.pem** (sirve para Windows 10/11, macOS y Linux con el comando `ssh`). Clic en **Create key pair**: el navegador descarga `clave-lab.pem`.

> **¿Por qué importa?** AWS **no guarda una copia** de tu clave privada. Si pierdes el archivo, no podrás entrar a esa instancia por SSH. Muévelo a una carpeta fija, por ejemplo `~/.ssh/` o `Documentos/aws-claves/`.

5 Configuración de red

Clic en **Edit** junto a **Network settings**. Deja la VPC por defecto y **Auto-assign public IP** en **Enable**. En **Firewall (security groups)** elige **Create security group** con nombre `sg-servidor-lab` y deja estas reglas:

Tipo	Puerto	Origen	Para qué
SSH	22	**My IP**	Solo tu conexión actual puede abrir una terminal.
HTTP	80	**Anywhere** (0.0.0.0/0)	Cualquiera puede ver tu sitio web (sesión 10).

> **¿Por qué "My IP" en SSH?** Dejar el puerto 22 abierto a `0.0.0.0/0` expone tu servidor a miles de intentos de acceso automáticos por hora. Limitarlo a tu IP elimina ese ruido y ese riesgo.

6 Almacenamiento

En **Configure storage** deja 8 GiB de tipo `gp3`. Alcanza de sobra para los laboratorios.

7 Revisa y lanza

En el panel **Summary** de la derecha verifica: 1 instancia, Ubuntu, `t3.micro`, `clave-lab`. Clic en **Launch instance** y luego en **View all instances**.

8 Espera a que esté lista

El estado pasa de `Pending` a `Running` en unos 30 segundos. Espera además a que la columna **Status check** muestre todas las comprobaciones superadas (por ejemplo `3/3 checks passed`). Selecciona la instancia y copia su **Public IPv4 address**.

> **Resultado esperado:** una instancia `servidor-lab` en estado `Running` con una IP pública del estilo `54.210.xx.xx`.

Conectarte por SSH desde tu computador

SSH exige que el archivo de la clave privada **solo pueda leerlo tu usuario**. Si otros usuarios del computador pueden leerlo, SSH se niega a usarlo. Ajusta los permisos según tu sistema:

macOS / Linux — Terminal — BASH

```bash
# Ve a la carpeta donde guardaste la clave
cd ~/Downloads

# Solo tu usuario puede leerla
chmod 400 clave-lab.pem

# Conéctate (cambia la IP por la tuya)
ssh -i clave-lab.pem ubuntu@54.210.10.20
```

Windows — PowerShell — BASH

```bash
# Ve a la carpeta donde guardaste la clave
cd $env:USERPROFILE\Downloads

# Quita la herencia y deja solo tu usuario
icacls.exe clave-lab.pem /inheritance:r
icacls.exe clave-lab.pem /grant:r "$($env:USERNAME):(R)"

# Conéctate (cambia la IP por la tuya)
ssh -i clave-lab.pem ubuntu@54.210.10.20
```

La primera vez, SSH te muestra la huella digital del servidor y pregunta si confías en él. Escribe `yes` y presiona `Enter`:

```
salida →
The authenticity of host '54.210.10.20' can't be established.
ED25519 key fingerprint is SHA256:x8Qm...
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-1012-aws x86_64)
ubuntu@ip-172-31-25-104:~$
```

Ese `ubuntu@ip-172-31-25-104:~$` es tu prompt: **ya estás dentro del servidor en Virginia**. Todo lo que escribas ahora se ejecuta allá, no en tu computador. Fíjate en que el nombre muestra la IP *privada* (`172.31.x.x`), no la pública; en la sesión 13 vas a entender por qué.

Explora tu servidor

dentro de la instancia — BASH

```bash
whoami                  # usuario actual → ubuntu
hostname                # nombre de la máquina
uname -a                # versión del kernel Linux
nproc                   # número de vCPU → 2
free -h                 # memoria RAM total y libre
df -h /                 # espacio en disco del volumen raíz
sudo apt update         # actualiza la lista de paquetes (sudo = como administrador)

# Pregúntale a la instancia sus propios datos (servicio de metadatos IMDSv2)
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 60")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-type
exit                    # cierra la sesión SSH y vuelve a tu computador
```

¿Sabías qué?

La dirección `169.254.169.254` solo existe dentro de cada instancia. Es el **servicio de metadatos**: le permite a un programa preguntar "¿qué tipo de máquina soy?, ¿en qué zona estoy?, ¿qué rol tengo?". La versión 2 (IMDSv2) exige primero pedir un token, lo que la protege de ataques.

Cuando SSH falla: diagnóstico rápido

Mensaje de error	Causa probable	Solución
`Connection timed out`	El grupo de seguridad no permite el puerto 22 desde tu IP (tu IP de casa o de la universidad cambió).	EC2 → Security Groups → `sg-servidor-lab` → Edit inbound rules → en la regla SSH vuelve a elegir **My IP**.
`Permission denied (publickey)`	Usuario incorrecto o clave equivocada.	Usa `ubuntu@` para Ubuntu y `ec2-user@` para Amazon Linux. Verifica que el `.pem` sea el de esa instancia.
`UNPROTECTED PRIVATE KEY FILE!`	Permisos demasiado abiertos en el `.pem`.	Repite `chmod 400` o los comandos `icacls`.
`Connection refused`	La instancia todavía está arrancando.	Espera a que las comprobaciones de estado estén en verde.
`Could not resolve hostname`	Error al escribir la IP.	Copia de nuevo la **Public IPv4 address**.

> ✅ **Alternativa sin archivo de clave: EC2 Instance Connect**
>
> En la consola, selecciona la instancia → **Connect** → pestaña **EC2 Instance Connect** → **Connect**, y se abre una terminal en el navegador. Ojo: el servicio se conecta desde un rango de IP de AWS, así que si tu regla SSH solo permite **My IP**, fallará. Para usarlo, agrega una regla SSH con el rango de EC2 Instance Connect de tu región (publicado en la documentación de AWS) o usa SSH desde tu terminal.

El ciclo de vida de una instancia

Acción	Estado resultante	¿Cobra cómputo?	¿Conserva el disco?	¿Conserva la IP pública?
Launch	`pending` → `running`	Sí	Sí	Recibe una nueva
Stop	`stopping` → `stopped`	No	Sí (sigue cobrando GB)	No: al encender recibe otra
Start	`pending` → `running`	Sí	Sí	Recibe una nueva
Reboot	sigue `running`	Sí	Sí	Sí
Terminate	`shutting-down` → `terminated`	No	No: se borra por defecto	No

> **! Terminate es irreversible**
>
> Una instancia terminada no se puede recuperar. Si tiene algo valioso, primero crea una copia del disco (`Actions → Image and templates → Create image`). Para instancias importantes activa `Termination protection` en `Actions → Instance settings` .

🧪 Laboratorio 4 — Tu servidor Linux en la nube

Objetivo: lanzar una instancia, conectarte por SSH, inspeccionarla y practicar el ciclo detener/iniciar.

`⏱ 35 min` `$ cubierto por el nivel gratuito` `Dificultad: media`

1 Lanza la instancia

Sigue los 8 pasos de "Lanzar tu primera instancia". Nombre `servidor-lab`, Ubuntu 24.04, tipo gratuito, clave `clave-lab`, grupo `sg-servidor-lab` con SSH (My IP) y HTTP (Anywhere).

2 Conéctate por SSH

Ajusta los permisos del `.pem` según tu sistema operativo y conéctate con `ssh -i clave-lab.pem ubuntu@TU_IP`.

3 Inspecciona el servidor

Ejecuta los comandos de "Explora tu servidor" y anota: número de vCPU, RAM total, espacio libre en disco y tipo de instancia según el servicio de metadatos.

4 Deja una huella

Ejecuta `echo "Hola desde la nube, $(date)" > saludo.txt` y luego `cat saludo.txt`. Sal con `exit`.

5 Detén e inicia la instancia

En la consola: `Instance state → Stop instance`. Cuando esté `stopped`, haz `Instance state → Start instance`. Compara la IP pública nueva con la anterior.

6 Comprueba que el disco se conservó

Conéctate con la **nueva** IP y ejecuta `cat saludo.txt`.

> **Resultado esperado:** la IP pública cambió, pero `saludo.txt` sigue ahí, porque el volumen EBS se conserva al detener. **No termines esta instancia**: la usarás en las sesiones 10, 12 y 14.

Reto extra: crea una **Elastic IP** (EC2 → Elastic IPs → Allocate) y asóciala a tu instancia. Detén e inicia de nuevo: ¿cambió la IP? Recuerda que una Elastic IP cobra por hora; libérala (`Disassociate` y luego `Release`) al terminar el reto si no la vas a seguir usando.

? Punto de control 09

Ayer te conectaste por SSH sin problema desde tu casa. Hoy, desde la biblioteca de la universidad, obtienes `Connection timed out`. La instancia está `Running` y no cambiaste nada. ¿Qué pasó y cómo lo arreglas sin abrir el puerto 22 a todo el mundo?

▼ **Ver respuesta**

La regla SSH del grupo de seguridad permite solo la IP de tu casa ("My IP" se guardó como una IP fija, por ejemplo `181.50.x.x/32`). En la biblioteca tu IP pública es otra. Edita la regla de entrada y vuelve a seleccionar `My IP` para registrar la IP actual (y elimina la anterior si ya no la necesitas).

SESIÓN 10 · PUBLICAR EN INTERNET

Tu primer sitio web en EC2: Nginx a mano y con user data

De una terminal vacía a una página visible para cualquier persona del mundo.

Un servidor web es un programa que escucha en el **puerto 80** (HTTP) o **443** (HTTPS) y responde con páginas cuando un navegador las pide. Vas a instalar **Nginx**, uno de los servidores web más usados del mundo, primero a mano y luego de forma automática.

Método 1: instalar Nginx a mano

dentro de servidor-lab — BASH

```bash
# 1. Actualiza la lista de paquetes e instala Nginx
sudo apt update
sudo apt install -y nginx

# 2. Verifica que el servicio esté activo
systemctl status nginx --no-pager

# 3. Pruébalo desde el propio servidor
curl -I http://localhost
```

```
salida →
● nginx.service - A high performance web server and a reverse proxy server
     Active: active (running) since ...
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
```

Ahora abre en tu navegador `http://TU_IP_PUBLICA` (con `http://`, no `https://`). Debes ver la página *"Welcome to nginx!"*. Esa petición viajó desde tu navegador, cruzó Internet, pasó el grupo de seguridad por el puerto 80 y llegó a Nginx.

Reemplaza la página por la tuya. Nginx sirve los archivos de la carpeta `/var/www/html`:

dentro de servidor-lab — BASH

```bash
sudo tee /var/www/html/index.html > /dev/null <<'HTML'
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Mi servidor en AWS</title></head>
<body style="font-family:sans-serif; text-align:center; padding:60px;">
  <h1>¡Hola desde AWS!</h1>
  <p>Esta página vive en una instancia EC2 en us-east-1.</p>
</body>
</html>
HTML
```

Recarga el navegador: ya ves tu página. `sudo tee` escribe en un archivo que requiere permisos de administrador, y el bloque `<<'HTML' ... HTML` (llamado *heredoc*) le pasa todo el texto de una sola vez.

! Si el navegador no carga la página

Revisa en orden: 1) ¿escribiste `http://` y no `https://`? (el navegador puede forzar HTTPS; escribe la URL completa). 2) ¿El grupo de seguridad tiene la regla HTTP 80 desde `0.0.0.0/0`? 3) ¿`systemctl status nginx` dice `active (running)`? 4) ¿Usas la IP pública *actual* (cambia si detuviste la instancia)?

Método 2: automatizar con user data

Instalar a mano sirve para aprender, pero si necesitas diez servidores idénticos no vas a repetir los comandos diez veces. El campo **user data** recibe un script que la instancia ejecuta **como root en su primer arranque**. Así la máquina nace ya configurada.

user-data.sh BASH

```
1 #!/bin/bash
2 apt-get update -y
  apt-get install -y nginx
3 TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 60")
  AZ=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/placement/availability-zone)
4 echo "<h1>Servidor automático</h1><p>Zona: $AZ</p>" > /var/www/html/index.html
5 systemctl enable --now nginx
```

1. **Shebang**: indica que el script se ejecuta con Bash. Es obligatorio en la primera línea.
2. Sin `sudo`: user data ya corre como root. `-y` responde "sí" automáticamente porque nadie va a estar ahí para escribirlo.
3. Consulta los metadatos para saber en qué zona de disponibilidad nació la instancia.
4. Genera una página que muestra esa zona. Muy útil para comprobar el balanceo entre zonas.
5. Arranca Nginx ahora y en cada reinicio.

Para usarlo: al lanzar una instancia, despliega **Advanced details**, baja hasta el final y pega el script en **User data**. Si algo falla, el registro de ejecución queda en `/var/log/cloud-init-output.log` dentro de la instancia.

✓ Infraestructura reproducible

Un servidor configurado a mano es una "mascota": único, difícil de reemplazar y nadie recuerda bien qué se le instaló. Un servidor creado por script es "ganado": si falla, lo reemplazas por uno idéntico en minutos. Los equipos profesionales llevan esta idea más lejos con herramientas de **infraestructura como código** como AWS CloudFormation o Terraform.

🧪 Laboratorio 5 — Dos servidores web: artesanal y automático

Objetivo: publicar una página a mano en `servidor-lab` y lanzar un segundo servidor que se configura solo con user data.

⏱ 30 min | $ cubierto por el nivel gratuito | Dificultad: media

1 Instala Nginx en servidor-lab

Conéctate por SSH e instala Nginx con el método 1.

2 Publica tu página personal

Reemplaza `index.html` con una página con tu nombre, tu carrera y una frase sobre lo que aprendiste. Ábrela desde tu celular usando datos móviles para confirmar que es pública.

3 Lanza un segundo servidor automático

Launch instances → nombre `servidor-auto`, Ubuntu 24.04, tipo gratuito, clave `clave-lab`, **grupo de seguridad existente** `sg-servidor-lab`. En **Advanced details → User data** pega el script `user-data.sh`.

4 Comprueba el resultado sin conectarte

Espera 2 o 3 minutos y abre `http://IP_DE_SERVIDOR_AUTO`.

> **Resultado esperado:** la página "Servidor automático" muestra la zona de disponibilidad (por ejemplo `us-east-1c`) sin que hayas escrito un solo comando en esa máquina.

5 Termina el servidor automático

Selecciona `servidor-auto` → **Instance state → Terminate (delete) instance**. Conserva `servidor-lab`.

Reto extra: modifica el script para que la página también muestre el ID de la instancia (`meta-data/instance-id`) y la hora de creación (`$(date)`). Lanza dos instancias a la vez (**Number of instances: 2**) y compara sus páginas. Termínalas al acabar.

? Punto de control 10

Pegaste un script de user data en una instancia que ya estaba corriendo desde hace una semana (lo editaste con la instancia detenida y la volviste a iniciar), pero Nginx no se instaló. ¿Por qué?

▼ **Ver respuesta**

Por defecto, user data se ejecuta **solo en el primer arranque** de la instancia. Reiniciar una instancia existente no lo vuelve a ejecutar. Para aplicarlo, lanza una instancia nueva o ejecuta los comandos a mano por SSH.

SESIÓN 11 · ALMACENAMIENTO DE OBJETOS

Amazon S3: buckets, objetos, permisos y sitios web estáticos

El servicio de almacenamiento más usado de Internet, explicado desde cero.

S3 (Simple Storage Service) guarda **objetos**: archivos de cualquier tipo, desde un texto de 1 byte hasta un video de 5 TB, cada uno con su propia dirección. No es un disco que montas en un servidor; es un servicio al que le hablas por HTTP: "guarda este archivo con este nombre" y "dame el archivo con este nombre".

La metáfora: una bodega de cajas etiquetadas

Un **bucket** es una bodega con un nombre único en el mundo. Dentro guardas cajas (**objetos**), cada una con una etiqueta única (**clave** o *key*). La bodega no tiene estanterías reales: si una etiqueta dice `fotos/2026/perro.jpg`, la consola te la muestra "como si" estuviera en carpetas, pero en realidad `fotos/2026/` es solo parte del nombre, llamado **prefijo**.

Concepto	Ejemplo	Reglas clave
Bucket	`fotos-tienda-jperez-2026`	Nombre único en **todo AWS**, de 3 a 63 caracteres, solo minúsculas, números, guiones y puntos. Vive en una región.
Objeto	El archivo `perro.jpg` + sus metadatos	Hasta 5 TB por objeto. Espacio total ilimitado.
Clave (key)	`fotos/2026/perro.jpg`	El nombre completo y único del objeto dentro del bucket.
URL del objeto	`https://fotos-tienda-jperez-2026.s3.us-east-1.amazonaws.com/fotos/2026/perro.jpg`	Existe siempre, pero por defecto responde `AccessDenied` a quien no tenga permiso.

¿Sabías qué?

S3 está diseñado para una durabilidad del **99,999999999 %** ("once nueves"): guarda copias en varios dispositivos de al menos tres zonas de disponibilidad. Estadísticamente, si guardas 10 millones de objetos, podrías perder uno cada 10 000 años. Por eso tantas empresas guardan ahí sus respaldos.

Clases de almacenamiento: paga según cuánto accedes

Clase	Cuándo usarla	Costo por GB
S3 Standard	Datos que se leen con frecuencia: imágenes de una web activa.	El más alto de almacenamiento, sin cobro por lectura adicional
S3 Intelligent-Tiering	No sabes con qué frecuencia se accede; AWS mueve los objetos solo.	Variable, más un pequeño cargo de monitoreo
S3 Standard-IA	Acceso infrecuente pero inmediato: reportes del mes pasado.	Menor almacenamiento, cobro por cada lectura
S3 Glacier (Instant, Flexible, Deep Archive)	Archivos históricos y respaldos que casi nunca se leen.	Muy bajo; recuperar puede tardar de milisegundos a horas

Seguridad: privado por defecto

Un bucket nuevo es **privado**: solo las identidades de tu cuenta con permisos de IAM pueden leerlo. Además trae activado **Block Public Access**, un interruptor de seguridad que impide que una política o configuración lo vuelva público por accidente. Los objetos se cifran automáticamente al guardarse.

> **! El error más famoso de la nube**
>
> Muchas filtraciones masivas de datos se originaron en buckets de S3 configurados como públicos por error. Regla: **solo desactiva Block Public Access en buckets creados exclusivamente para contenido público** (como un sitio web estático), y nunca guardes ahí nada privado. Para compartir un archivo privado de forma temporal, usa una **URL prefirmada**.

Paso a paso: crear un bucket y subir archivos

`Consola › S3 › Buckets › Create bucket`

1 Tipo y nombre

Tipo `General purpose`. Nombre único, por ejemplo `archivos-lab-` seguido de tus iniciales y un número: `archivos-lab-jp-4821`. Verifica que la región sea `us-east-1`.

2 Propiedad de objetos

Deja `ACLs disabled (recommended)`. Así todos los permisos se manejan con políticas y no con listas de control antiguas.

3 Bloqueo de acceso público

Deja marcada **Block all public access**.

4 Versionado

Activa **Bucket Versioning: Enable**. S3 guardará cada versión de un archivo cuando lo sobrescribas o lo borres, para que puedas recuperarlo.

5 Cifrado y creación

Deja el cifrado por defecto (`SSE-S3`) y haz clic en **Create bucket**.

6 Sube un archivo

Entra al bucket → **Upload** → **Add files** → elige una imagen → **Upload**.

7 Intenta abrirlo por su URL

Haz clic en el objeto y copia su **Object URL** en una ventana de incógnito.

> **Resultado esperado:** un XML con `<Code>AccessDenied</Code>`. Eso es **correcto**: tu bucket es privado. En cambio, el botón **Open** de la consola sí te lo muestra, porque genera un enlace temporal firmado con tus credenciales.

Lo mismo desde la CLI

CloudShell o terminal — BASH

```bash
BUCKET=archivos-lab-jp-4821                    # cambia por el nombre de tu bucket

echo "Informe de laboratorio" > informe.txt
aws s3 cp informe.txt s3://$BUCKET/documentos/informe.txt   # subir un archivo
aws s3 ls s3://$BUCKET --recursive                          # listar todo el contenido
aws s3 cp s3://$BUCKET/documentos/informe.txt copia.txt     # descargar

mkdir -p sitio && echo "hola" > sitio/a.txt
aws s3 sync ./sitio s3://$BUCKET/sitio/                     # sincronizar una carpeta completa

# Enlace temporal de 10 minutos a un objeto privado
aws s3 presign s3://$BUCKET/documentos/informe.txt --expires-in 600
```

salida →

```
upload: ./informe.txt to s3://archivos-lab-jp-4821/documentos/informe.txt
https://archivos-lab-jp-4821.s3.us-east-1.amazonaws.com/documentos/informe.txt?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=...&X-Amz-Expires=600&X-Amz-Signature=...
```

Abre esa URL prefirmada en una ventana de incógnito: funciona durante 10 minutos y luego responde `AccessDenied`. Así comparten las apps profesionales facturas o documentos privados sin volver público el bucket.

Sitio web estático en S3

Un sitio **estático** es aquel formado solo por archivos HTML, CSS, JavaScript e imágenes, sin código de servidor: un portafolio, una landing page o una app de React ya compilada. S3 puede servirlo directamente, sin ningún servidor EC2.

Para que cualquiera pueda leer los archivos, el bucket necesita una **política de bucket** que permita `s3:GetObject` a todo el mundo:

politica-bucket-publico.json JSON

```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "LecturaPublicaDelSitio",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::mi-portafolio-jp-4821/*"
    }
  ]
}
```

1. **`Principal "*"`**: cualquier persona, sin autenticarse. Las políticas de bucket, a diferencia de las de IAM, indican *a quién* se aplican.
2. Solo lectura de objetos. Nadie puede listar, subir ni borrar.
3. `/*` = todos los objetos del bucket. **`Cambia el nombre por el de tu bucket`** o la política fallará.

🧪 **Laboratorio 6 — Publica tu portafolio en S3**

Objetivo: crear un bucket público exclusivo para un sitio estático y publicarlo con una URL propia de S3.

⏱ 30 min | $ centavos o cubierto por el nivel gratuito | Dificultad: media

1 Crea el bucket del sitio

Nombre `mi-portafolio-` + iniciales + número. Región `us-east-1`. En esta ocasión **desmarca** **Block all public access** y marca la casilla de confirmación que aparece. Crea el bucket.

> **¿Por qué ahora sí?** Este bucket existe *solo* para contenido que quieres que todo el mundo vea. Jamás subas aquí nada privado.

2 Crea tu página

En tu computador crea `index.html` con tu nombre, una foto y enlaces a tus proyectos, y `error.html` con un mensaje "Página no encontrada".

3 Súbelos

Bucket → **Upload** → agrega ambos archivos → **Upload**.

4 Activa el hosting estático

Pestaña **Properties** → baja hasta **Static website hosting** → **Edit** → **Enable**. Index document: `index.html`. Error document: `error.html`. Guarda.

5 Agrega la política de lectura pública

Pestaña **Permissions** → **Bucket policy** → **Edit**. Pega la política de arriba con el nombre de **tu** bucket en `Resource` y guarda.

6 Abre tu sitio

Vuelve a **Properties → Static website hosting** y copia el **Bucket website endpoint**, de la forma `http://mi-portafolio-jp-4821.s3-website-us-east-1.amazonaws.com`.

> **Resultado esperado:** tu portafolio se ve desde cualquier dispositivo. Si escribes una ruta inexistente (`/nada`), aparece tu `error.html`.

Reto extra: modifica `index.html` y súbelo con `aws s3 sync . s3://TU_BUCKET` desde la carpeta del proyecto. Luego, con el versionado activado en el bucket de la sección anterior, sobrescribe `informe.txt` dos veces y usa el interruptor **Show versions** para recuperar la primera versión.

✓ Para producción: CloudFront delante de S3

El endpoint de sitio web de S3 solo sirve HTTP, no HTTPS. En un proyecto real pones **CloudFront** delante del bucket: obtienes HTTPS con certificado gratuito, caché en ubicaciones de borde de todo el mundo y puedes mantener el bucket privado. Es el siguiente paso natural cuando domines este laboratorio.

? Punto de control 11

Tu app debe permitir que cada cliente descargue *su* factura en PDF, y solo la suya. Un compañero propone hacer público el bucket de facturas "porque las URLs son difíciles de adivinar". ¿Qué le respondes y qué propones?

▼ **Ver respuesta**

Que es un antipatrón: una URL "difícil de adivinar" termina filtrándose (en historiales, correos reenviados o registros), y un bucket público puede ser listado por error de configuración. Propón mantener el bucket **privado** y que el backend, después de verificar que el usuario es dueño de la factura, genere una **URL prefirmada** que caduque en pocos minutos.

SESIÓN 12 · BASES DE DATOS GESTIONADAS

Amazon RDS: una base de datos PostgreSQL sin administrar servidores

Tú escribes SQL; AWS se encarga de instalar, parchar y respaldar.

Podrías instalar PostgreSQL dentro de tu instancia EC2 con `apt install`. Funciona... hasta que tienes que configurar copias de seguridad diarias, aplicar parches de seguridad, monitorear el disco, restaurar datos después de un error o montar una réplica en otra zona. **RDS** (Relational Database Service) hace todo eso por ti. Es un servicio **gestionado**: tú administras los datos y las consultas; AWS administra el motor y la máquina.

Tarea	PostgreSQL instalado en EC2	Amazon RDS
Instalación y actualizaciones	Tú, a mano	Automáticas, en la ventana que elijas
Copias de seguridad	Tú configuras scripts y dónde guardarlas	Diarias y automáticas, con restauración a un punto en el tiempo
Alta disponibilidad	Configuración compleja de replicación	Un clic: Multi-AZ
Acceso al sistema operativo	Sí (control total)	No (no hay SSH a la máquina de RDS)
Costo	Solo la instancia EC2	Algo más caro por el trabajo que te ahorra

Motores disponibles

RDS ofrece **PostgreSQL, MySQL, MariaDB, Oracle, SQL Server y Db2**, además de **Amazon Aurora**, un motor propio de AWS compatible con PostgreSQL y MySQL, más rápido y escalable pero más costoso. En este

capítulo usamos **PostgreSQL**: es de código abierto, muy completo y el más valorado en la industria para proyectos nuevos.

Conceptos de disponibilidad y respaldo

Concepto	Qué hace	Cuándo lo usas
Respaldos automáticos	Copia diaria + registros de transacciones. Permiten restaurar a cualquier segundo dentro del período de retención (1 a 35 días).	Siempre. Te salva de un `DELETE` sin `WHERE`.
Snapshot manual	Una "foto" de la base que se guarda hasta que tú la borres.	Antes de un cambio grande o antes de eliminar la base.
Multi-AZ	Una copia en espera en otra zona de disponibilidad, sincronizada en todo momento. Si la principal falla, RDS cambia a la copia en uno o dos minutos.	Producción. Duplica el costo; no lo actives en laboratorios.
Réplica de lectura	Una copia que acepta solo consultas `SELECT`.	Cuando muchas lecturas saturan la base principal.

Paso a paso: crear la base de datos

```
Consola › RDS › Databases › Create database
```

1 **Método de creación**

Elige **Standard create** (en consolas recientes puede aparecer como **Full configuration**) para ver y entender cada opción.

2 **Motor**

Selecciona **PostgreSQL** y deja la versión que aparece por defecto.

3 **Plantilla**

Elige **Free tier** (o **Sandbox**). Esta plantilla bloquea automáticamente las opciones costosas como Multi-AZ.

4 Identificador y credenciales

DB instance identifier: `bd-tienda`. Master username: `postgres`. En **Credentials management** elige **Self managed** y escribe una contraseña fuerte (mínimo 8 caracteres, sin `/`, `"`, `@` ni espacios). Guárdala en tu gestor de contraseñas.

> **Alternativa profesional:** la opción **Managed in AWS Secrets Manager** genera y rota la contraseña por ti. Es lo recomendado en producción, pero Secrets Manager tiene un costo mensual por secreto.

5 Tipo de instancia y almacenamiento

Instancia: `db.t4g.micro` o `db.t3.micro`. Almacenamiento: 20 GiB. Despliega **Additional storage configuration** y **desmarca** **Enable storage autoscaling** para que el disco no crezca solo.

6 Conectividad (el paso más importante)

En **Compute resource** elige **Connect to an EC2 compute resource** y selecciona `servidor-lab`. En **Public access** deja **No**.

> **¿Qué hace esta opción?** AWS crea dos grupos de seguridad enlazados: uno para la instancia y otro para la base, con una regla que dice "el puerto 5432 de la base solo acepta conexiones que vengan de `servidor-lab`". Tu base queda invisible para Internet y alcanzable solo desde tu backend. En la sesión 13 entenderás exactamente cómo funciona.

7 Nombre de la base inicial

Despliega **Additional configuration** y en **Initial database name** escribe `tienda`. Si lo dejas vacío, RDS no crea ninguna base de datos dentro de la instancia. Deja la retención de respaldos en 1 día.

8 Crea y espera

Revisa el recuadro **Estimated monthly costs** y haz clic en **Create database**. El estado pasa por `Creating` y `Backing-up` hasta llegar a `Available`, en 5 a 15 minutos.

9 Copia el endpoint

Haz clic en `bd-tienda` → pestaña **Connectivity & security**. Copia el **Endpoint** (algo como `bd-tienda.c9xyzabc1234.us-east-1.rds.amazonaws.com`) y confirma que el puerto es `5432`.

> **Resultado esperado:** base `bd-tienda` en estado `Available`, sin acceso público, con un endpoint que vas a usar desde tu instancia EC2.

Conectarte desde tu instancia EC2 con psql

dentro de servidor-lab — BASH

```bash
# Instala el cliente de PostgreSQL (no el servidor)
sudo apt install -y postgresql-client

# Guarda el endpoint en una variable para no repetirlo
export DB_HOST=bd-tienda.c9xyzabc1234.us-east-1.rds.amazonaws.com

# Conéctate exigiendo cifrado SSL (te pedirá la contraseña)
psql "host=$DB_HOST port=5432 dbname=tienda user=postgres sslmode=require"
```

```
salida →
psql (16.x, server 17.x)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
tienda⟹
```

El prompt `tienda⟹` significa que ya estás dentro de la base de datos. Crea la tabla que usará tu backend en la sesión 14:

psql — base tienda

SQL

```sql
-- Tabla de productos de la tienda
CREATE TABLE productos (
  id          SERIAL PRIMARY KEY,          -- número que se asigna solo: 1, 2, 3...
  nombre      VARCHAR(100) NOT NULL,
  precio      NUMERIC(10,2) NOT NULL,      -- hasta 99 999 999,99
  creado_en   TIMESTAMP DEFAULT NOW()
);

INSERT INTO productos (nombre, precio) VALUES
  ('Teclado mecánico', 189000),
  ('Mouse inalámbrico', 65000),
  ('Monitor 24 pulgadas', 720000);

SELECT id, nombre, precio FROM productos ORDER BY precio DESC;
```

salida →

```
 id |       nombre        |  precio
----+---------------------+-----------
  3 | Monitor 24 pulgadas | 720000.00
  1 | Teclado mecánico    | 189000.00
  2 | Mouse inalámbrico   |  65000.00
(3 rows)
```

Sal de `psql` con `\q`. Comandos útiles dentro de `psql`: `\dt` lista las tablas, `\d productos` describe la tabla y `\l` lista las bases de datos.

! Antipatrón: base de datos pública "para poder conectarme desde mi portátil"

Poner `Public access: Yes` y abrir el puerto 5432 a `0.0.0.0/0` expone tu base a ataques de fuerza bruta de todo Internet. Si necesitas administrarla desde tu computador, conéctate a través de la instancia EC2 (un túnel SSH, por ejemplo con `ssh -L 5432:ENDPOINT:5432 ubuntu@IP`) o usa RDS Query Editor cuando esté disponible para tu motor.

🧪 **Laboratorio 7 — La base de datos de la tienda**

Objetivo: crear una base PostgreSQL privada en RDS, conectarte desde EC2 y dejar lista la tabla de productos.

⏱ 40 min (incluye 10 de espera) | $ cubierto por el nivel gratuito | Dificultad: media

1 Crea la base

Sigue los 9 pasos de "Crear la base de datos". Mientras esperas el estado `Available`, lee la sesión 13.

2 Instala el cliente y conéctate

Desde `servidor-lab`, instala `postgresql-client` y conéctate con `psql` y `sslmode=require`.

3 Crea y llena la tabla

Ejecuta el script SQL. Agrega al menos dos productos más con tus propios nombres y precios.

4 Practica consultas

Escribe consultas que: a) muestren los productos de más de 100 000; b) cuenten cuántos productos hay (`SELECT COUNT(*) FROM productos;`); c) actualicen el precio de uno con `UPDATE ... WHERE id = ...`.

5 Comprueba que la base es privada

Desde **tu computador** (no desde EC2) intenta `ping` o una conexión al endpoint.

> **Resultado esperado:** desde EC2 todo funciona; desde tu computador la conexión no responde (o el nombre resuelve a una IP privada `172.31.x.x`). Eso demuestra que la base está protegida dentro de la VPC.

Reto extra: crea un snapshot manual (**Actions → Take snapshot** , nombre `tienda-antes-de-borrar`). Luego ejecuta `DELETE FROM productos WHERE id = 1;` y piensa: ¿cómo recuperarías ese producto usando los respaldos automáticos o el snapshot? (Pista: se restaura como una *nueva* instancia de base de datos). Borra el snapshot al terminar el capítulo.

? Punto de control 12

Un compañero dice: "RDS es lo mismo que instalar PostgreSQL en EC2, solo que más caro". Da dos argumentos técnicos que justifiquen pagar RDS en un proyecto real.

▼ **Ver respuesta**

1) **Respaldos automáticos con restauración a un punto en el tiempo**, sin escribir ni vigilar scripts. 2) **Alta disponibilidad Multi-AZ** con conmutación automática en otra zona. También valen: parches automáticos, monitoreo integrado y escalado del almacenamiento sin detener la base.

SESIÓN 13 · REDES EN LA NUBE

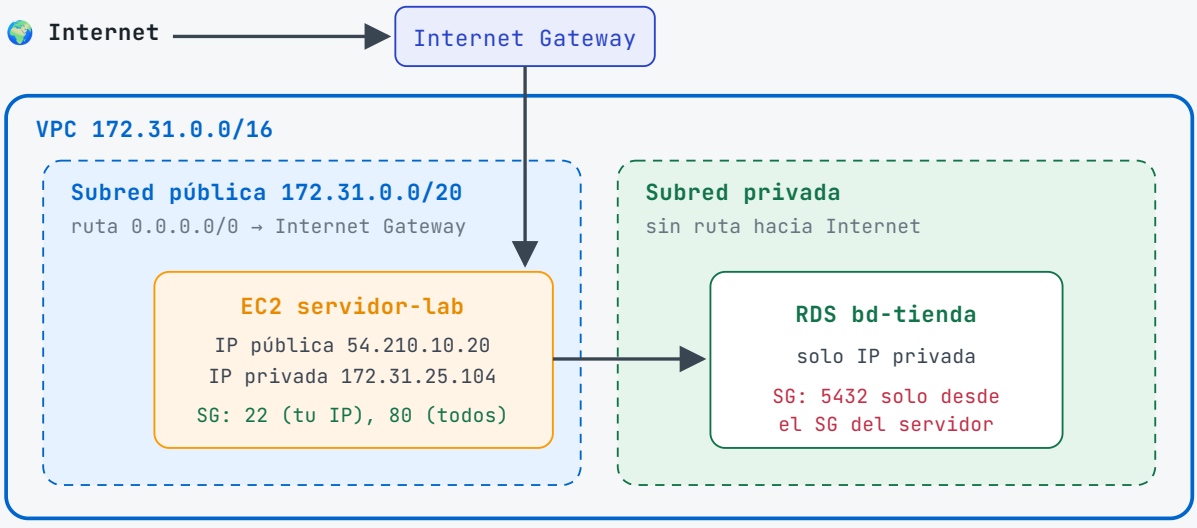
VPC, subredes, tablas de rutas y grupos de seguridad

Por qué tu servidor web es visible y tu base de datos no.

Una **VPC** (Virtual Private Cloud) es tu red privada dentro de AWS, aislada de las redes de los demás clientes. Todo lo que creaste (EC2, RDS) vive dentro de una VPC. Cada cuenta trae una **VPC por defecto** en cada región, lista para usar; por eso no tuviste que configurar redes hasta ahora.

La metáfora: un conjunto residencial cerrado

La **VPC** es el conjunto completo con su muro. Las **subredes** son las torres dentro del conjunto: unas dan a la calle (públicas) y otras están al fondo (privadas). El **Internet Gateway** es la portería principal: la única puerta hacia la calle. Las **tablas de rutas** son los letreros que dicen por dónde se sale desde cada torre. Y los **grupos de seguridad** son el citófono de cada apartamento: deciden a quién se le abre.



El tráfico de Internet entra por el Internet Gateway solo hasta la subred pública. La base de datos, en la subred privada, únicamente recibe conexiones del servidor web.

Direcciones IP y notación CIDR

Cada recurso en la VPC tiene una **IP privada**. Los rangos de direcciones se escriben en notación **CIDR**: una IP base, una barra y un número que indica cuántos bits son fijos. Cuantos más bits fijos, menos direcciones disponibles.

CIDR	Bits fijos	Direcciones totales	Uso típico
`10.0.0.0/16`	16	65 536	Una VPC completa
`10.0.1.0/24`	24	256 (AWS reserva 5 → 251 usables)	Una subred
`54.210.10.20/32`	32	1	Una sola IP, como tu regla "My IP"
`0.0.0.0/0`	0	Todas las de Internet	"Cualquier origen" o "cualquier destino"

La fórmula es **$2^{(32 - \text{bits fijos})}$**: un `/24` tiene $2^8 = 256$ direcciones. En cada subred AWS reserva cinco direcciones para su propio uso (red, enrutador, DNS, uso futuro y difusión).

¿Qué hace pública a una subred?

No es una casilla que dice "pública". Una subred es pública **si su tabla de rutas tiene una ruta hacia un Internet Gateway**:

tabla de rutas de la subred pública — RUTAS

```
Destino          Objetivo            Significado
172.31.0.0/16    local               tráfico dentro de la VPC se queda en la VPC
0.0.0.0/0        igw-0a1b2c3d4e      todo lo demás sale por el Internet Gateway
```

Una subred privada tiene solo la ruta `local`. Si sus servidores necesitan descargar actualizaciones de Internet sin ser alcanzables desde afuera, se agrega un **NAT Gateway** (una puerta de solo salida). Ojo: el NAT Gateway se cobra por hora y por GB, incluso en el nivel gratuito, así que no lo crees en tus laboratorios salvo que lo necesites.

Grupos de seguridad frente a Network ACL

Característica	Grupo de seguridad (SG)	Network ACL (NACL)
Se aplica a	Cada instancia o recurso (su interfaz de red)	Toda una subred
Tipos de regla	Solo **permitir**	Permitir y **denegar**
Estado	**Con estado**: si entra una petición permitida, la respuesta sale automáticamente	**Sin estado**: debes permitir explícitamente la entrada y la salida
Evaluación	Todas las reglas juntas	En orden numérico; la primera que coincide gana
Uso habitual	La herramienta principal del día a día	Una capa extra, por ejemplo para bloquear una IP maliciosa

La técnica clave: un grupo de seguridad que referencia a otro

En la regla de entrada de la base de datos no pusiste una IP, sino **otro grupo de seguridad** como origen. Eso significa: "acepta en el puerto 5432 a cualquier recurso que tenga el grupo del servidor web". Si mañana lanzas tres servidores más con ese grupo, todos podrán conectarse a la base sin que edites nada; y nadie más podrá hacerlo.

reglas de entrada de los grupos de seguridad — SG

```
sg-servidor-lab (servidor web)
  Tipo    Puerto  Origen
  SSH     22      181.50.23.7/32      ← solo tu IP
  HTTP    80      0.0.0.0/0           ← todo Internet

rds-ec2-1 (base de datos)
  Tipo        Puerto  Origen
  PostgreSQL  5432    sg-0f1e2d3c...      ← solo recursos del grupo del servidor web
```

✓ Arquitectura en capas (defensa en profundidad)

Internet solo toca el puerto 80 del servidor web. El servidor web es el único que toca la base. La base nunca toca Internet. Si un atacante encuentra una falla en una capa, todavía tiene que superar las siguientes. Así se diseñan los sistemas profesionales.

🧪 Laboratorio 8 — Detective de redes

Objetivo: recorrer la VPC por defecto y explicar, con evidencia de la consola, por qué el servidor es público y la base es privada.

⏱ 25 min | $ 0 USD | Dificultad: media

1 Encuentra tu VPC

`Consola › VPC › Your VPCs` Anota el ID y el CIDR de la VPC por defecto (`172.31.0.0/16`).

2 Cuenta las subredes

Menú **Subnets**. ¿Cuántas hay? Anota en qué zona de disponibilidad está cada una y su CIDR (`/20`). ¿Cuántas direcciones tiene cada una?

3 Lee la tabla de rutas

Menú **Route tables** → la tabla principal → pestaña **Routes**. Encuentra la ruta `0.0.0.0/0` y el ID del Internet Gateway al que apunta.

4 Rastrea los grupos de seguridad

EC2 → **Security Groups**. Abre el grupo que RDS creó para la base (su nombre empieza por `rds-ec2-`) y lee su regla de entrada. Compara el ID de origen con los grupos asignados a `servidor-lab` (pestaña **Security** de la instancia).

5 Rompe algo a propósito (y arréglalo)

Quita temporalmente la regla HTTP 80 de `sg-servidor-lab` y recarga tu página: deja de cargar. Vuelve a agregarla: carga de nuevo. Los cambios en grupos de seguridad son inmediatos.

> **Resultado esperado:** puedes dibujar en una hoja el diagrama de esta sesión con los datos reales de tu cuenta: IDs, CIDR, zonas y reglas.

Reto extra: usa el asistente **Create VPC → VPC and more** solo para *ver* la vista previa (sin crearla) de una VPC con 2 subredes públicas y 2 privadas en 2 zonas. Observa en el diagrama cómo se conectan las tablas de rutas. Luego cancela.

? Punto de control 13

a) ¿Cuántas direcciones tiene un bloque `/20` ? **b)** Tu grupo de seguridad permite la entrada HTTP en el puerto 80 pero no tiene ninguna regla de salida especial. ¿Llegan las respuestas al navegador del usuario? ¿Por qué?

▼ **Ver respuesta**

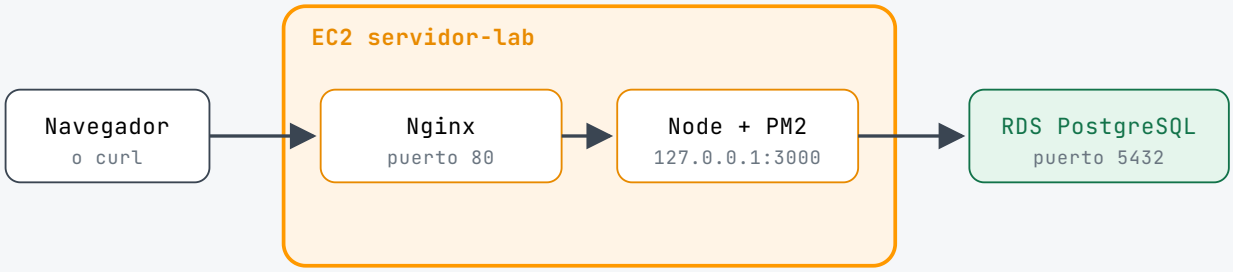
a) $2^{(32-20)} = 2^{12} =$ **4096** direcciones. **b)** Sí. Los grupos de seguridad son **con estado**: si permiten una petición de entrada, la respuesta sale automáticamente sin necesitar una regla de salida. (Además, por defecto un grupo nuevo permite todo el tráfico de salida).

SESIÓN 14 · INTEGRACIÓN COMPLETA

Desplegar una API Node.js con Express, PostgreSQL, PM2 y Nginx

Todo lo aprendido, conectado en una aplicación real que responde en Internet.

Llegó el momento de unir las piezas. Vas a desplegar una API REST de la tienda que lista y crea productos en la base de RDS. La arquitectura final queda así:



Nginx recibe el tráfico público y lo reenvía a Node, que escucha solo en la interfaz local. Node consulta la base de datos privada.

Pieza	Qué hace	Por qué la necesitas
Express	Framework web de Node.js que define las rutas de la API.	Responde a `GET /productos` y `POST /productos`.
pg	Cliente de PostgreSQL para Node.js.	Permite que tu código consulte RDS.
dotenv	Carga variables de entorno desde un archivo `.env`.	Mantiene la contraseña fuera del código fuente.
PM2	Gestor de procesos para Node.js.	Mantiene la app viva, la reinicia si falla y la arranca cuando la instancia se reinicia.
Nginx	Servidor web que actúa como **proxy inverso**.	Expone el puerto 80 al mundo; más adelante agrega HTTPS, compresión y límites de tráfico.

Paso 1: instalar Node.js en el servidor

Los repositorios de Ubuntu traen versiones antiguas de Node. Usa el repositorio oficial de NodeSource para instalar una versión LTS (soporte de largo plazo):

dentro de servidor-lab — BASH

```bash
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs
node --version      # debe mostrar v22.x.x
npm --version
```

Paso 2: crear el proyecto e instalar dependencias

dentro de servidor-lab — BASH

```bash
mkdir -p ~/api-tienda && cd ~/api-tienda
npm init -y                          # crea package.json
npm install express pg dotenv        # instala las tres dependencias

# Certificado de las autoridades de AWS para verificar la conexión cifrada con RDS
curl -fsSL -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
```

Paso 3: configuración con variables de entorno

Crea el archivo `.env` con `nano .env`, pega el contenido con tus datos, guarda con `Ctrl` + `O`, `Enter` y sal con `Ctrl` + `X`:

.env — ENV

```env
# Conexión a la base de datos RDS (nunca subas este archivo a Git)
DB_HOST=bd-tienda.c9xyzabc1234.us-east-1.rds.amazonaws.com
DB_PORT=5432
DB_NAME=tienda
DB_USER=postgres
DB_PASSWORD=TuContraseñaSegura
PORT=3000
```

dentro de servidor-lab — BASH

```bash
chmod 600 .env                    # solo tu usuario puede leer el archivo
echo ".env" >> .gitignore         # por si luego subes el proyecto a GitHub
echo "node_modules" >> .gitignore
```

Paso 4: el código de la API

Crea `server.js` con `nano server.js` y escribe el siguiente código. Lee cada comentario: explica qué hace cada bloque.

server.js — JS

```js
require('dotenv').config();              // carga las variables del archivo .env
const fs = require('fs');
const express = require('express');
const { Pool } = require('pg');

const app = express();
app.use(express.json());                 // permite leer cuerpos JSON en las peticiones

const pool = new Pool({
  host: process.env.DB_HOST,
  port: Number(process.env.DB_PORT),
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  ssl: { ca: fs.readFileSync('global-bundle.pem').toString() }   // conexión cifrada y verificada
```

```
});

// Ruta de salud: comprueba que la API y la base respondan
app.get('/salud', async (req, res) => {
  try {
    await pool.query('SELECT 1');
    res.json({ estado: 'ok', baseDeDatos: 'conectada' });
  } catch (err) {
    res.status(500).json({ estado: 'error', detalle: err.message });
  }
});

// Listar todos los productos
app.get('/productos', async (req, res) => {
  const { rows } = await pool.query('SELECT id, nombre, precio FROM productos ORDER BY id');
  res.json(rows);
});

// Crear un producto nuevo
app.post('/productos', async (req, res) => {
  const { nombre, precio } = req.body;
  if (!nombre || precio === undefined) {
    return res.status(400).json({ error: 'nombre y precio son obligatorios' });
  }
  const { rows } = await pool.query(
    'INSERT INTO productos (nombre, precio) VALUES ($1, $2) RETURNING *',
    [nombre, precio]
  );
  res.status(201).json(rows[0]);          // 201 = recurso creado
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, '127.0.0.1', () => {
  console.log(`API de la tienda escuchando en http://127.0.0.1:${PORT}`);
});
```

1. **dotenv** se carga primero, para que `process.env` ya tenga los valores cuando se cree la conexión.
2. **Pool de conexiones**: en vez de abrir una conexión nueva por cada petición (lento), reutiliza varias conexiones abiertas. La opción `ssl.ca` cifra el tráfico y verifica que realmente hablas con RDS.
3. **/salud**: una ruta de diagnóstico. Los balanceadores de carga y los sistemas de monitoreo la consultan para saber si la app está viva.

4. **`Validación de entrada`**: nunca confíes en los datos del cliente. Si faltan campos, responde `400 Bad Request` y no toques la base.
5. **`Consulta parametrizada`**: `$1` y `$2` se reemplazan de forma segura por los valores del arreglo. **Jamás** armes SQL concatenando texto del usuario (`"... VALUES ('" + nombre + "')"`): eso abre la puerta a la *inyección SQL*.
6. **`127.0.0.1`**: Node escucha solo dentro de la máquina. Nadie de Internet puede llegar al puerto 3000 directamente; todo pasa por Nginx.

> **Errores en rutas asíncronas**
>
> `npm install express` instala Express 5, que captura automáticamente los errores de las funciones `async` y responde con un error 500 en lugar de dejar la petición colgada. Si trabajas con Express 4, debes envolver cada ruta en `try/catch` como en `/salud`.

Paso 5: probar la API dentro del servidor

dentro de servidor-lab — BASH

```bash
node server.js &                    # ejecuta la API en segundo plano
curl http://127.0.0.1:3000/salud
curl http://127.0.0.1:3000/productos
kill %1                             # detén la prueba antes de pasar a P
M2
```

```
salida →
API de la tienda escuchando en http://127.0.0.1:3000
{"estado":"ok","baseDeDatos":"conectada"}
[{"id":1,"nombre":"Teclado mecánico","precio":"189000.00"},{"id":2,"nombre":"Mouse
inalámbrico","precio":"65000.00"}, ...]
```

¿Notaste que el precio llega como texto (`"189000.00"`)? El driver `pg` devuelve los `NUMERIC` como cadenas para no perder precisión decimal. Si necesitas números en el frontend, conviértelos con cuidado o usa enteros (centavos) en la base.

Paso 6: mantener la API viva con PM2

Si cierras la sesión SSH, un proceso lanzado con `node server.js` muere. PM2 lo convierte en un servicio permanente:

dentro de servidor-lab — BASH

```bash
sudo npm install -g pm2                    # instala PM2 de forma global
pm2 start server.js --name api-tienda      # arranca la API como proceso administrado
pm2 status                                 # tabla con estado, CPU y memoria
pm2 logs api-tienda --lines 20             # últimos mensajes de la app

pm2 startup                                # imprime un comando con sudo: cópialo y ejecútalo
pm2 save                                   # guarda la lista de procesos para restaurarla al reiniciar
```

salida →

```
┌────┬─────────────┬──────┬────┬──────────┬──────────┐
│ id │ name        │ mode │ ↺  │ status   │ memory   │
├────┼─────────────┼──────┼────┼──────────┼──────────┤
│ 0  │ api-tienda  │ fork │ 0  │ online   │ 52.1mb   │
└────┴─────────────┴──────┴────┴──────────┴──────────┘
```

Paso 7: Nginx como proxy inverso

Reemplaza la configuración del sitio por defecto de Nginx para que reenvíe todo al puerto 3000. Abre el archivo con `sudo nano /etc/nginx/sites-available/default`, borra su contenido y pega:

/etc/nginx/sites-available/default — NGINX

```nginx
server {
    listen 80;                          # puerto público
    server_name _;                      # responde a cualquier nombre o IP

    location / {
        proxy_pass http://127.0.0.1:3000;   # reenvía a la API de Node
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

dentro de servidor-lab

BASH

```bash
sudo nginx -t                  # valida la sintaxis antes de aplicar
sudo systemctl reload nginx    # aplica la configuración sin cortar conexiones
```

salida →

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Paso 8: probar desde tu computador

tu computador (macOS, Linux o Windows con curl.exe)

BASH

```bash
# Listar productos (también puedes abrir esta URL en el navegador)
curl http://54.210.10.20/productos

# Crear un producto (en Windows PowerShell escribe curl.exe en lugar de curl)
curl -X POST http://54.210.10.20/productos -H "Content-Type: application/json" -d "{\"nombre\":\"Audífonos\",\"precio\":120000}"

# Probar la validación: debe responder 400
curl -i -X POST http://54.210.10.20/productos -H "Content-Type: application/json" -d "{}"
```

salida →

```
{"id":4,"nombre":"Audífonos","precio":"120000.00"}
HTTP/1.1 400 Bad Request
{"error":"nombre y precio son obligatorios"}
```

Cómo actualizar la aplicación

En un flujo real no escribes el código con `nano` en el servidor: lo desarrollas en tu computador, lo subes a GitHub y en el servidor haces `git pull`. Otra opción rápida es copiar la carpeta con `scp`:

tu computador

BASH

```bash
# Copia el archivo modificado desde tu computador al servidor
scp -i clave-lab.pem ./api-tienda/server.js ubuntu@54.210.10.20:~/api-tienda/

# En el servidor, reinicia la app para cargar los cambios
ssh -i clave-lab.pem ubuntu@54.210.10.20 "pm2 restart api-tienda"
```

! Antes de mostrarla al mundo: HTTPS

Tu API funciona por HTTP, así que los datos viajan sin cifrar. Para producción necesitas un dominio propio (por ejemplo, registrado en Route 53) apuntando a una Elastic IP, y un certificado TLS gratuito de Let's Encrypt instalado con `certbot --nginx`. También deberías abrir el puerto 443 en el grupo de seguridad.

Diagnóstico cuando algo falla

Síntoma	Dónde mirar	Causa frecuente
`502 Bad Gateway` en el navegador	`pm2 status` y `pm2 logs api-tienda`	La app de Node está caída o escucha en otro puerto.
`/salud` responde error de conexión o *timeout*	Grupos de seguridad de RDS y la variable `DB_HOST`	Endpoint mal copiado o la base no permite el grupo de seguridad del servidor.
`password authentication failed`	Archivo `.env`	Contraseña o usuario incorrectos (cuidado con espacios al final).
`relation "productos" does not exist`	`psql` → `\dt`	No creaste la tabla o estás conectado a otra base (revisa `DB_NAME`).
El navegador no carga nada	Grupo de seguridad y `sudo nginx -t`	Falta la regla HTTP 80 o Nginx tiene un error de sintaxis.

🧪 **Laboratorio 9 — Tu API en producción (versión estudiante)**

Objetivo: desplegar la API de la tienda en EC2, conectada a RDS, administrada por PM2 y publicada por Nginx.

⏱ 60 min | $ cubierto por el nivel gratuito | Dificultad: alta

1 Prepara el servidor

Instala Node.js 22 (paso 1) y crea el proyecto con sus dependencias y el certificado de RDS (paso 2).

2 Configura los secretos

Crea `.env` con tus datos reales y protégelo con `chmod 600` (paso 3).

3 Escribe el código

Crea `server.js` (paso 4). Escríbelo tú en lugar de copiarlo: así detectas qué entiendes y qué no.

4 Prueba local

Ejecuta las pruebas del paso 5. No avances hasta que `/salud` diga `"conectada"`.

5 Hazla permanente

Arráncala con PM2, ejecuta el comando que imprime `pm2 startup` y luego `pm2 save` (paso 6).

6 Publica con Nginx

Configura el proxy inverso, valida con `nginx -t` y recarga (paso 7).

7 Prueba desde fuera

Desde tu computador lista productos, crea dos nuevos y prueba la validación (paso 8).

8 Prueba de resistencia

Reinicia la instancia desde la consola (**Instance state → Reboot**). Espera un minuto y vuelve a consultar `/productos` sin conectarte por SSH.

> **Resultado esperado:** después del reinicio la API responde sola, porque PM2 y Nginx arrancan con el sistema. Los productos que creaste siguen ahí, porque viven en RDS, no en el servidor.

Reto extra: agrega las rutas `GET /productos/:id` (responde 404 si no existe) y `DELETE /productos/:id`, ambas con consultas parametrizadas. Despliégalas con `scp` y `pm2 restart`.

? Punto de control 14

a) ¿Por qué Node escucha en `127.0.0.1` y no en `0.0.0.0`? **b)** Si un usuario envía como nombre `'); DROP TABLE productos; --`, ¿qué pasa con tu código? ¿Y si hubieras concatenado el texto en el SQL?

▼ **Ver respuesta**

a) Para que el puerto 3000 solo sea accesible desde la propia máquina: todo el tráfico externo pasa obligatoriamente por Nginx, que es la única puerta pública. **b)** Con la consulta parametrizada, el texto se guarda literalmente como el nombre de un producto y no se ejecuta. Si lo hubieras concatenado, PostgreSQL lo interpretaría como código SQL y podría borrar la tabla completa: eso es una **inyección SQL**.

SESIÓN 15 · SIN SERVIDORES

AWS Lambda: código que se ejecuta solo cuando lo necesitas

Sin instancias, sin parches, sin pagar por tiempo ocioso.

Tu servidor EC2 está encendido las 24 horas, aunque reciba una petición por hora. **Lambda** cambia el modelo: subes una **función** y AWS la ejecuta solo cuando ocurre un **evento** (una petición HTTP, un archivo nuevo en S3, un horario). Pagas por número de invocaciones y por milisegundos de ejecución. Si nadie la llama, pagas cero.

i ”Serverless” no significa ”sin servidores”

Los servidores existen, pero **no son tu problema**: AWS los aprovisiona, los escala de cero a miles de ejecuciones simultáneas y los apaga. Tú solo escribes la función.

★ ¿Sabías qué?

Lambda incluye una cuota gratuita mensual permanente de un millón de invocaciones. Para muchos proyectos personales y tareas automáticas, el costo real es literalmente cero.

EC2 frente a Lambda

Criterio	EC2	Lambda
Cobro	Por tiempo encendido, se use o no	Por invocación y milisegundos de ejecución
Administración	Tú: sistema operativo, parches, procesos	AWS: todo excepto tu código
Duración máxima	Ilimitada	15 minutos por ejecución
Escalado	Configuras Auto Scaling	Automático e inmediato
Arranque	Siempre listo	La primera llamada tras un tiempo inactiva tarda un poco más (*cold start*)
Ideal para	Apps con tráfico constante, procesos largos, WebSockets	APIs con tráfico variable, tareas por eventos, automatizaciones

Anatomía de una función

index.mjs — JS

```
1 export const handler = async (event) => {
2   const nombre = event.queryStringParameters?.nombre ?? 'mundo';
3
4   console.log('Petición recibida para:', nombre);
5
6   return {
7     statusCode: 200,
8     headers: { 'Content-Type': 'application/json' },
9     body: JSON.stringify({
10      mensaje: `Hola, ${nombre}. Te saluda Lambda.`,
11      hora: new Date().toISOString()
12    })
13  };
14 };
```

1. **`handler`**: la función que Lambda invoca. Recibe el `event` con los datos de quien la llamó.
2. Lee el parámetro `?nombre=` de la URL. `?.` y `??` evitan errores si no viene.
3. Todo `console.log` queda guardado en **`CloudWatch Logs`**.
4. La respuesta HTTP: código de estado, encabezados y cuerpo como texto JSON.

🧪 Laboratorio 10 — Tu primera función con URL pública

Objetivo: crear una función Lambda, probarla en la consola, publicarla con una Function URL y revisar sus registros.

⏱ 25 min | $ 0 USD | Dificultad: baja

1 Crea la función

`Consola › Lambda › Functions › Create function` Elige **Author from scratch**.

Nombre: `saludo-api`. Runtime: **Node.js 22.x** (o la versión más reciente que aparezca). Arquitectura: **arm64** (más económica). Clic en **Create function**.

> **¿Qué pasó detrás?** Lambda creó automáticamente un **rol de IAM** para la función, con permiso para escribir sus registros en CloudWatch. Así aplica el mínimo privilegio desde el primer minuto.

2 Escribe el código

En el editor, abre `index.mjs`, reemplaza su contenido por el código anterior y haz clic en **Deploy**.

3 Pruébala en la consola

Pestaña **Test** → crea un evento llamado `prueba` con el JSON `{"queryStringParameters": {"nombre": "Ana"}}` → **Test**. Lee la respuesta, la duración y la memoria usada.

4 Publícala con una Function URL

Pestaña **Configuration** → **Function URL** → **Create function URL**. Auth type: **NONE** (pública). Guarda y copia la URL, que termina en `.lambda-url.us-east-1.on.aws`.

5 Llámala desde el navegador

Abre `https://TU_URL/?nombre=TuNombre`. Fíjate que ya viene con **HTTPS**, sin configurar nada.

6 Lee los registros

Pestaña **Monitor** → **View CloudWatch logs** → abre el flujo de registros más reciente y encuentra tu mensaje "Petición recibida para".

> **Resultado esperado:** un JSON con tu saludo y la hora, servido por HTTPS, sin ningún servidor que administrar, y el registro de cada llamada en CloudWatch.

Reto extra: agrega un parámetro `idioma` que devuelva el saludo en español, inglés o portugués. Luego, en **Add trigger**, explora (sin crearlo) el disparador de S3: ¿qué evento haría que la función se ejecute sola al subir una imagen?

! Una URL pública sin autenticación es pública de verdad

Cualquiera que tenga la URL puede invocar la función las veces que quiera. Para un saludo no importa; para una función que escribe en una base de datos o llama servicios pagos, protégela con `AWS_IAM` o ponla detrás de **API Gateway** con autenticación y límites de peticiones. Y bórrala al terminar el laboratorio.

? Punto de control 15

Para cada caso, ¿EC2 o Lambda? **a)** Generar una miniatura cada vez que un usuario sube una foto. **b)** Un servidor de videojuego multijugador con conexiones abiertas durante horas. **c)** Enviar un reporte por correo todos los lunes a las 7:00.

▼ Ver respuesta

a) Lambda, disparada por el evento de S3. **b)** EC2 (o contenedores): las conexiones duran más de 15 minutos y el tráfico es constante. **c)** Lambda, programada con Amazon EventBridge Scheduler: se ejecuta unos segundos a la semana y no paga por el resto del tiempo.



SESIÓN 16 · OPERAR COMO PROFESIONAL

Monitoreo, limpieza total y próximos pasos

Vigila lo que construiste, borra lo que ya no usas y planea tu siguiente nivel.

CloudWatch: los ojos de tu infraestructura

Amazon CloudWatch recolecta tres tipos de información: **métricas** (números en el tiempo, como el porcentaje de CPU), **registros** (los mensajes que imprimen tus aplicaciones y servicios) y **alarmas** (reglas

que te avisan cuando una métrica cruza un umbral). EC2 envía métricas básicas cada 5 minutos sin costo adicional.

`Consola › CloudWatch › Alarms › Create alarm`

1 **Elige la métrica**

Select metric → **EC2** → **Per-Instance Metrics** → busca el ID de `servidor-lab` y marca `CPUUtilization`. Clic en **Select metric**.

2 **Define la condición**

Estadística: **Average**. Período: **5 minutes**. Condición: **Greater** que `80`.

3 **Configura la notificación**

En **Notification** elige **Create new topic**, nombre `alertas-lab`, escribe tu correo y haz clic en **Create topic**. Esto usa Amazon SNS, el servicio de notificaciones.

4 **Nombra y crea la alarma**

Nombre: `cpu-alta-servidor-lab`. Clic en **Create alarm**.

5 **Confirma la suscripción**

Revisa tu correo y haz clic en **Confirm subscription** en el mensaje de AWS Notifications. Sin ese clic no te llegará ninguna alerta.

> **Resultado esperado:** la alarma aparece con estado `OK` (o `Insufficient data` los primeros minutos).

Reto: dispara tu propia alarma

En `servidor-lab` ejecuta `timeout 900 yes > /dev/null &` dos veces (una por cada vCPU). Eso pone la CPU al 100 % durante 15 minutos. Espera 10 minutos: la alarma debe pasar a `In alarm` y deberías recibir un correo. Los procesos se detienen solos al cumplirse el tiempo.

La limpieza total: el hábito que te separa de un susto

Cuando termines el capítulo (o cada vez que acabes una práctica que no vas a continuar), elimina los recursos **en este orden**. El orden importa: algunos recursos dependen de otros y no se pueden borrar mientras estén en uso.

Laboratorio 11 — Limpieza total de la cuenta

Objetivo: dejar la cuenta sin recursos que generen costos y verificarlo en la facturación.

`⏱ 25 min` `$ ahorra dinero` `Dificultad: baja, pero obligatoria`

1 Elimina la base de datos RDS

RDS → selecciona `bd-tienda` → **Actions → Delete**. Desmarca **Create final snapshot** (salvo que quieras conservar los datos), desmarca **Retain automated backups**, marca la casilla de confirmación, escribe `delete me` y confirma. En **Snapshots** borra también los snapshots manuales que hayas creado.

2 Termina las instancias EC2

EC2 → Instances → selecciona `servidor-lab` y cualquier otra → **Instance state → Terminate (delete) instance**.

3 Libera las Elastic IP

EC2 → Elastic IPs → si hay alguna: **Actions → Release Elastic IP addresses**. Una Elastic IP sin usar sigue cobrando.

4 Revisa volúmenes y snapshots de EBS

EC2 → Volumes: no debe quedar ninguno en estado `available`. EC2 → Snapshots y AMIs: borra los que hayas creado.

5 Vacía y elimina los buckets de S3

S3 → selecciona cada bucket → **Empty** (escribe `permanently delete`) → luego **Delete** (escribe el nombre del bucket). Con la CLI: `aws s3 rb s3://NOMBRE --force`. Si activaste el versionado, **Empty** borra también todas las versiones.

6 Elimina la función Lambda y su URL

Lambda → `saludo-api` → **Actions → Delete**.

7 Borra alarmas, temas SNS y grupos de registros

CloudWatch → Alarms → borra `cpu-alta-servidor-lab`. SNS → Topics → borra `alertas-lab`. CloudWatch → Log groups → borra los de `/aws/lambda/saludo-api`.

8 Limpia grupos de seguridad y pares de claves

EC2 → Security Groups: borra `sg-servidor-lab` y los `rds-ec2-*` / `ec2-rds-*` (no borres el grupo `default`). EC2 → Key Pairs: borra `clave-lab` y elimina también el archivo `.pem` de tu computador.

9 Verifica en todas las regiones

Busca **EC2 Global View** en la consola: muestra instancias, volúmenes y VPC de **todas** las regiones en una sola pantalla. Luego abre **Billing and Cost Management → Bills** y confirma que no aparezcan servicios activos inesperados.

> **Resultado esperado:** EC2 Global View sin instancias, S3 sin buckets del laboratorio, RDS sin bases y la factura del mes sin cargos nuevos. Conserva tu usuario IAM, el MFA y los presupuestos: son gratuitos y seguirán protegiéndote.

Reto extra: escribe un script de Bash que use la CLI para listar en `us-east-1` todas las instancias EC2, los buckets S3, las bases RDS y las funciones Lambda (`aws ec2 describe-instances`, `aws s3 ls`, `aws rds describe-db-instances`, `aws lambda list-functions`). Úsalo como lista de verificación al final de cada práctica.

Antipatrones frente a buenas prácticas

❌ Antipatrón	✅ Buena práctica
Trabajar a diario con el usuario raíz.	Raíz con MFA guardado; trabajo diario con un usuario IAM con MFA.
Crear recursos sin alertas de gasto.	Presupuestos configurados antes del primer recurso.
SSH (22) abierto a `0.0.0.0/0`.	SSH limitado a tu IP, o acceso sin puertos abiertos con Session Manager.
Base de datos con acceso público.	Base privada que solo acepta el grupo de seguridad del backend.
Contraseñas y claves escritas en el código o en GitHub.	Variables de entorno, `.gitignore`, Secrets Manager y roles de IAM.
Bucket público "porque es más fácil".	Bucket privado y URL prefirmadas; público solo para sitios estáticos.
Todo en una sola zona de disponibilidad.	Recursos críticos repartidos en al menos dos zonas.
Configurar servidores a mano y no documentar.	User data, scripts e infraestructura como código.
SQL armado concatenando texto del usuario.	Consultas parametrizadas y validación de entrada.
Dejar recursos encendidos "por si acaso".	Apagar o eliminar al terminar; revisar la factura cada semana.

El marco de buena arquitectura (Well-Architected)

AWS resume décadas de experiencia en seis **pilares** para evaluar cualquier arquitectura. Cuando diseñes un sistema, pregúntate cómo cumple cada uno:

Excelencia operativa
Automatiza, monitorea y mejora los procesos continuamente.

Seguridad
Mínimo privilegio, cifrado y trazabilidad en todas las capas.

Fiabilidad
Recuperarse de fallas: varias zonas, respaldos y pruebas.

Eficiencia de rendimiento
El tipo y tamaño de recurso adecuado para cada carga.

Optimización de costos
Pagar solo por lo que aporta valor; eliminar lo ocioso.

Sostenibilidad
Reducir el consumo de energía y recursos de tu carga de trabajo.

Glosario esencial

AMI	Plantilla con sistema operativo y software desde la que se lanza una instancia EC2.
ARN	Identificador único de un recurso de AWS: `arn:aws:servicio:región:cuenta:recurso`.
AZ	Zona de disponibilidad: uno o más centros de datos aislados dentro de una región.
Bucket	Contenedor de objetos en S3, con nombre único global.
CIDR	Notación para rangos de IP, por ejemplo `10.0.0.0/16`.
CloudWatch	Servicio de métricas, registros y alarmas.
EBS	Volumen de almacenamiento en bloque (disco) para instancias EC2.
EC2	Servicio de máquinas virtuales (instancias).
Elastic IP	Dirección IPv4 pública fija que puedes asociar a una instancia.
IAM	Servicio de identidades y permisos: usuarios, grupos, roles y políticas.
Internet Gateway	Componente que conecta una VPC con Internet.
Lambda	Servicio de funciones serverless que se ejecutan por eventos.
MFA	Autenticación de múltiples factores: contraseña + un código o dispositivo.
NAT Gateway	Permite que subredes privadas salgan a Internet sin ser alcanzables desde fuera.
Política	Documento JSON que permite o niega acciones sobre recursos.
Proxy inverso	Servidor (como Nginx) que recibe peticiones públicas y las reenvía a una aplicación interna.
RDS	Servicio de bases de datos relacionales gestionadas.
Región	Área geográfica con varias zonas de disponibilidad, por ejemplo `us-east-1`.
Rol	Identidad de IAM sin credenciales permanentes que se asume temporalmente.
S3	Servicio de almacenamiento de objetos.
Security Group	Firewall virtual con estado que controla el tráfico de un recurso.
Subred	Segmento de una VPC ubicado en una sola zona de disponibilidad.

`URL prefirmada`	Enlace temporal que da acceso a un objeto privado de S3.
`User data`	Script que una instancia EC2 ejecuta en su primer arranque.
`VPC`	Red virtual privada y aislada dentro de AWS.

Proyecto final: la tienda completa

Reto integrador

Reconstruye desde cero, sin mirar el paso a paso, una versión ampliada de la tienda y documenta el proceso en un repositorio de GitHub (sin secretos):

1. **Backend en EC2** creado con un script de user data que instale Node.js, Nginx y PM2.
2. **Base RDS privada** con tablas `productos` y `pedidos` relacionadas por clave foránea.
3. **Imágenes de productos en S3**, privadas; la API devuelve una URL prefirmada de 5 minutos para cada imagen.
4. **Rol de IAM para la instancia** con permiso solo para leer ese bucket, sin claves de acceso en el servidor.
5. **Una función Lambda** que responda con estadísticas básicas (por ejemplo, número de productos).
6. **Alarma de CloudWatch** de CPU con notificación por correo.
7. **README** con el diagrama de arquitectura, los grupos de seguridad y su justificación, el costo estimado mensual según la calculadora de AWS y las instrucciones de limpieza.

Criterio de éxito: otra persona puede leer tu README, entender la arquitectura y explicar por qué la base de datos no es accesible desde Internet.

Tu ruta de aprendizaje después de este capítulo

Etapa	Qué aprender	Meta concreta
1. Consolidar	Repetir los laboratorios sin guía; roles de IAM para EC2; HTTPS con un dominio.	Desplegar tu proyecto personal con HTTPS.
2. Certificarte	Fundamentos, facturación, seguridad y servicios principales.	AWS Certified Cloud Practitioner.
3. Escalar	Load Balancer, Auto Scaling, CloudFront, DynamoDB, API Gateway.	Una app en dos zonas que escale sola.
4. Automatizar	Contenedores (Docker, ECS), infraestructura como código (CloudFormation, CDK o Terraform), CI/CD.	Desplegar con un `git push`.
5. Especializarte	Arquitectura, seguridad, datos o inteligencia artificial en la nube.	AWS Certified Solutions Architect – Associate.

Recursos oficiales gratuitos: **AWS Skill Builder** (`skillbuilder.aws`) con cursos y laboratorios, la documentación oficial (`docs.aws.amazon.com`) y el programa **AWS Educate** para estudiantes.

📌 Lo que ya sabes hacer

- Explicar qué es la nube, sus modelos y el modelo de responsabilidad compartida.
- Crear, blindar y vigilar financieramente una cuenta de AWS.
- Gestionar permisos con IAM y operar AWS desde la CLI.
- Lanzar y administrar servidores EC2, publicar sitios en S3 y crear bases en RDS.
- Diseñar una red segura en capas con subredes y grupos de seguridad.
- Desplegar una API real, crear funciones serverless, monitorear y limpiar.

? Punto de control 16 (repaso general)

Un mes después de terminar el capítulo recibes un correo de AWS Budgets: tu gasto superó los 5 USD. Describe, en orden, los pasos que sigues para encontrar y detener la causa.

▼ Ver respuesta

1) Entrar a **Billing → Bills** y ver qué servicio y qué **región** generan el cobro. 2) Ir a ese servicio en esa región (o usar **EC2 Global View**) y localizar el recurso. 3) Detenerlo o eliminarlo (instancia, base de datos, Elastic IP, NAT Gateway, volumen o snapshot olvidado). 4) Si aparecen recursos que *no creaste*, asumir credenciales comprometidas: desactivar claves de acceso, cambiar contraseñas, revisar usuarios de IAM y contactar al soporte de AWS. 5) Volver a revisar la factura al día siguiente para confirmar que el gasto se detuvo.

Ya construiste en la nube

Empezaste sin cuenta y terminaste con una arquitectura de tres capas funcionando en Internet: un servidor web, una base de datos privada, almacenamiento de objetos y una función serverless, todo protegido con permisos mínimos y alertas de costos. Eso es exactamente lo que hacen los equipos de ingeniería todos los días, a mayor escala.

Siguiente capítulo sugerido

Alta disponibilidad: Load Balancer, Auto Scaling y CloudFront

Los nombres de botones, planes, precios y cuotas gratuitas de AWS cambian con frecuencia. Verifica siempre la información vigente en `aws.amazon.com/free`, `aws.amazon.com/pricing` y `docs.aws.amazon.com` antes de crear recursos.